

IXChange BV

# SDR-SLC

## SLC-CP Control Plane Specification

| Field        | Value                                                                      |
|--------------|----------------------------------------------------------------------------|
| Document     | SDR-SLC-CP-1.0                                                             |
| Title        | SLC-CP Control Plane Specification                                         |
| Version      | Protocol 1.0   Release 1.3 (transmit controls, timestamps, VITA alignment) |
| Status       | RELEASE CANDIDATE                                                          |
| Date         | September 2026                                                             |
| Author       | Ivo van Ling, PA2IX                                                        |
| Organisation | IXChange BV                                                                |
| Licence      | Creative Commons CC BY 4.0 — Open Source                                   |

Copyright © 2026 IXChange BV. All rights reserved. Published under Creative Commons CC BY 4.0.

# Table of Contents

|                                                           |    |
|-----------------------------------------------------------|----|
| <b>Revision History</b>                                   | 5  |
| <b>1. Introduction</b>                                    | 5  |
| 1.1 What changed in Release 1.3, and why                  | 6  |
| 1.2 Breaking changes                                      | 6  |
| <b>2. Normative Language</b>                              | 7  |
| <b>3. Device Model: Envelope and Function Blocks</b>      | 7  |
| 3.1 Why the envelope is exactly these two commands        | 7  |
| 3.2 Function blocks defined in this release               | 8  |
| 3.3 Combining blocks in one device                        | 8  |
| 3.4 Function identifiers                                  | 8  |
| 3.5 The vendor prefix rule                                | 8  |
| <b>4. Implementation Profiles</b>                         | 8  |
| 4.1 Block rx                                              | 8  |
| 4.2 Block tx                                              | 9  |
| 4.3 Block codec                                           | 9  |
| 4.4 Declaring a tier                                      | 9  |
| <b>5. Building a Minimal SDR-SLC Device</b>               | 9  |
| 5.1 The complete Tier 1 receive conversation              | 9  |
| 5.2 What a minimal device may omit                        | 11 |
| 5.3 How a minimal device interoperates with a full client | 11 |
| 5.4 Startup sequence                                      | 12 |
| <b>6. Discovery</b>                                       | 12 |
| 6.1 The TXT field functions replaces role                 | 12 |
| 6.2 The fw field names the implementation                 | 12 |
| 6.3 A codec service has its own service type              | 12 |
| 6.4 One device that does both                             | 13 |
| <b>7. Terminology</b>                                     | 13 |
| <b>8. Transport and Framing [ENVELOPE]</b>                | 14 |
| 8.1 TCP connection                                        | 14 |
| 8.2 Message encoding                                      | 14 |
| 8.3 Length-prefixed framing                               | 14 |
| 8.4 Field conventions                                     | 14 |
| <b>9. Message Model [ENVELOPE]</b>                        | 14 |
| 9.1 Request                                               | 15 |
| 9.2 Successful response                                   | 15 |
| 9.3 Error response                                        | 15 |
| 9.4 Asynchronous event                                    | 15 |
| 9.5 Response correlation                                  | 15 |
| <b>10. Version Negotiation [ENVELOPE]</b>                 | 16 |
| 10.1 hello                                                | 16 |
| 10.2 ping [ANY • required at RX-T2 and CODEC-T2]          | 17 |
| <b>11. Capability Exchange [ENVELOPE]</b>                 | 17 |
| 11.1 Structure                                            | 17 |
| 11.2 Schema versioning                                    | 17 |
| 11.3 Control descriptors                                  | 17 |
| <b>12. Function Addressing</b>                            | 18 |
| 12.1 Why this replaced direction                          | 19 |
| 12.2 Direction within a function                          | 19 |
| 12.3 Channels                                             | 19 |
| 12.4 Shared sample clock between blocks                   | 20 |
| 12.5 Connectors                                           | 20 |
| <b>13. Configuration Commands</b>                         | 21 |
| 13.1 set_frequency [RX-T1 • TX-T1]                        | 21 |
| 13.2 set_sample_rate [RX-T1 • TX-T1]                      | 21 |
| 13.3 set_control [RX-T1 • RX-T2 • TX-T1]                  | 21 |
| 13.4 get_control [RX-T2]                                  | 22 |
| <b>14. Stream Session Management</b>                      | 23 |

|                                                                       |           |
|-----------------------------------------------------------------------|-----------|
| 14.1 open_stream [RX-T1 · TX-T1 · CODEC-T1].....                      | 23        |
| 14.2 Why transport parameters are grouped.....                        | 23        |
| 14.3 Advertising transports.....                                      | 23        |
| 14.4 Ports: who binds what.....                                       | 24        |
| 14.5 The vita object for a transmit stream.....                       | 24        |
| 14.6 signal_type.....                                                 | 24        |
| 14.7 start_rx / stop_rx [RX-T1].....                                  | 25        |
| 14.8 close_stream [RX-T1 · TX-T1 · CODEC-T1].....                     | 25        |
| <b>15. Function Block: tx.....</b>                                    | <b>25</b> |
| 15.1 Capabilities.....                                                | 25        |
| 15.2 get_tx_capabilities [TX-T1].....                                 | 26        |
| 15.3 start_tx / stop_tx [TX-T1].....                                  | 26        |
| 15.4 PTT and turnaround.....                                          | 26        |
| 15.5 set_tx_power [TX-T2].....                                        | 26        |
| 15.6 get_vswr and auto_tune [TX-T3].....                              | 27        |
| 15.7 Transmit safety.....                                             | 27        |
| 15.8 get_timestamps [TX-T2].....                                      | 28        |
| <b>16. Function Block: codec.....</b>                                 | <b>28</b> |
| 16.1 Formats and conversions.....                                     | 29        |
| 16.2 Opening a conversion.....                                        | 30        |
| 16.3 RTP endpoints and port binding.....                              | 30        |
| 16.4 Chaining conversions.....                                        | 30        |
| 16.5 Interoperability profile for existing vocoder servers [ANY]..... | 31        |
| 16.6 Why JSON negotiation and not SDP.....                            | 31        |
| 16.7 What the codec block does not cover.....                         | 31        |
| <b>17. Access Control and Ownership.....</b>                          | <b>31</b> |
| 17.1 The distinction that drives the model.....                       | 31        |
| 17.2 Declaring an access model.....                                   | 31        |
| 17.3 Reporting the current state.....                                 | 32        |
| 17.4 Listening in is a feature.....                                   | 32        |
| 17.5 Capacity.....                                                    | 32        |
| <b>18. Status Reporting [RX-T2 · CODEC-T2].....</b>                   | <b>32</b> |
| 18.1 Response structure.....                                          | 32        |
| 18.2 Behaviour.....                                                   | 33        |
| 18.3 hardware_monitors [RX-T2].....                                   | 33        |
| <b>19. Reference Level and Power Display [RX-T2].....</b>             | <b>34</b> |
| 19.1 Tier 2 capability hints.....                                     | 34        |
| <b>20. Clock and Location [RX-T2 query · RX-T3 set].....</b>          | <b>35</b> |
| 20.1 Location [RX-T3].....                                            | 35        |
| <b>21. Asynchronous Events.....</b>                                   | <b>35</b> |
| 21.1 Naming and scope.....                                            | 35        |
| 21.2 Event registry.....                                              | 35        |
| 21.3 adc_overload.....                                                | 36        |
| 21.4 monitor_alert.....                                               | 36        |
| 21.5 Subscriptions.....                                               | 37        |
| <b>22. Error Code Registry [ENVELOPE].....</b>                        | <b>37</b> |
| <b>23. Extensibility and Forward Compatibility [ENVELOPE].....</b>    | <b>38</b> |
| 23.1 Reserved field names.....                                        | 38        |
| <b>24. Conformance Requirements.....</b>                              | <b>39</b> |
| 24.1 Envelope — every device.....                                     | 39        |
| 24.2 Block rx, Tier 1.....                                            | 39        |
| 24.3 Block rx, Tier 2.....                                            | 39        |
| 24.4 Block tx, Tier 1.....                                            | 40        |
| 24.5 Block tx, Tier 2.....                                            | 40        |
| 24.6 Block codec, Tier 1.....                                         | 40        |
| 24.7 Clients, any tier.....                                           | 41        |
| <b>25. Security Considerations.....</b>                               | <b>41</b> |
| 25.1 Ownership is not a security mechanism.....                       | 41        |
| 25.2 A network of services increases exposure.....                    | 41        |

|                                                    |           |
|----------------------------------------------------|-----------|
| 25.3 Transmitters.....                             | 42        |
| 25.4 Future work.....                              | 42        |
| <b>26. Authentication [ANY].....</b>               | <b>42</b> |
| 26.1 Authentication is declared per function.....  | 42        |
| 26.2 Method: psk-hmac-sha256.....                  | 42        |
| 26.3 Behaviour.....                                | 43        |
| 26.4 What this protects, and what it does not..... | 43        |
| 26.5 Extending the method set.....                 | 44        |
| <b>27. References.....</b>                         | <b>44</b> |
| 27.1 Normative references.....                     | 44        |
| 27.2 Informative references.....                   | 44        |

## Revision History

| Rev         | Date       | Author      | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|-------------|------------|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1.0 draft   | 2026-05-01 | I. van Ling | Initial draft. Core command set established.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| 1.1 draft   | 2026-05-16 | I. van Ling | TX commands, turnaround_mode, conformance tiers.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| 1.2 draft   | 2026-05-20 | I. van Ling | Typed control system, AGC, bias_tee, PPM, direct_sampling, preferred hints.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| 1.3 draft   | 2026-05-21 | I. van Ling | signal_type in open_stream. VITA-49 Context packet client behaviour.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| 1.4 draft   | 2026-05-22 | I. van Ling | Reference Level calibration in capabilities, get_status, set_control trigger rule.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| 1.5 draft   | 2026-05-22 | I. van Ling | adc_overload, adc_level_dbfs, lo_locked, actual_sample_rate_sps, hardware_monitors, monitor_alert event, bias_tee float redesign, reserved field names.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| 1.0 release | 2026-05-22 | I. van Ling | Consolidated release. Three-tier structure.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| 1.1 release | 2026-05-23 | I. van Ling | Revised for implementers. Four implementation profiles. Restored implementation-guide narrative and full command detail. Added "Building a Minimal SDR-SLC Device". TX no longer requires VSWR, GPS, or hardware monitors.                                                                                                                                                                                                                                                                                                                                                                                                                       |
| 1.2 release | 2026-07-29 | I. van Ling | Function block model. Envelope separated from the receive function. direction renamed to function. Tiers per function block. codec block with RTP transport. Access control and ownership. Optional per-function authentication. Transmit safety timeout. Multi-channel receivers. Transport parameters grouped per transport. Discovery TXT field role replaced by functions.                                                                                                                                                                                                                                                                   |
| 1.3 release | 2026-09-14 | I. van Ling | Alignment release, from building the two reference implementations and the macOS client against 1.2. Transmit-side controls made explicit. turnaround_mode moved into the tx block. set_tx_power fully specified. Client side of open_stream for tx defined. signal_type restored. get_timestamps [TX-T2] for scheduled bursts. stream_state carries reason. Shared sample clock declared. Free-running clock rule aligned with SDR-SLC-VITA-1.0 Release 1.2. Editorial corrections throughout.                                                                                                                                                  |
| 1.3 release | 2026-09-16 | I. van Ling | Amendment folded into Release 1.3 before publication. Connectors (§12.5): a block or channel with more than one physical port declares connectors [ ], and a control descriptor names the connector it belongs to; antenna reserved control. iq_formats MAY be declared per channel. board_temperature_celsius, supply_voltage_v, total_current_ma promoted to standard hardware_monitors channels; usable_bandwidth_hz promoted to a defined hint. Zero-point rule for unsigned encodings in the power formula. tx Tier 2 requires set_tx_power or rf_gain; turnaround_mode optional on full-duplex hardware; tx nullable status fields listed. |

### WARNING

Release 1.3 changes the wire format in the places listed in §1.2 and is not compatible with implementations built against Release 1.2. This document remains a RELEASE CANDIDATE; Protocol 1.0 has never been declared final, and a release candidate is not a stable interface. Once Protocol 1.0 FINAL is published, the compatibility rules in §23 apply without exception.

## 1. Introduction

The SDR-SLC project exists because modern networked software-defined radio is technically mature but practically inaccessible. The equipment that does it well is expensive, runs on a single operating system, and speaks proprietary protocols. This project aims to fix all three of those problems.

SLC-CP is the control plane for SDR-SLC devices. It is a JSON protocol over TCP. JSON was chosen deliberately: it is human-readable, you can debug it with netcat, and every programming language from MicroPython to Swift can parse it. The control plane tells a device what to do; the data itself travels over a separate transport chosen to suit the medium.

The hardware side of SDR-SLC is designed for home-builders. The reference design uses an STM32H7 microcontroller and a Tayloe quadrature detector for the RF front end, streams I/Q samples over 100BASE-T Ethernet, and can be built for under €100 in components. An RTL-SDR dongle bridged to this protocol via a Raspberry Pi daemon is also a fully conformant implementation.

The protocol is intentionally layered and tiered. You do not have to implement everything to be compatible. A minimal receiver is nine commands and takes about a weekend. More capable hardware adds more features without breaking simpler clients.

This document is both a normative specification and a practical implementation handbook. Every command section explains what the command does, why it exists, what you must implement, and how to implement it on constrained hardware. Read the whole document once, then use it as a reference.

|             |                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>NOTE</b> | The SDR-SLC protocol suite consists of five documents. This document covers the control plane only. The other four are: SDR-SLC-PS-1.0 — Protocol Suite Architecture (read this first for context). SDR-SLC-DISC-1.0 — Service Discovery (mDNS/DNS-SD, two service types). SDR-SLC-VITA-1.0 — I/Q Stream Transport (VITA-49, receive and transmit profiles). SDR-SLC-RTP-1.0 — Framed Media Transport (RTP payload formats for the codec function block). |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Two open-source reference implementations exist and are named in this document where their behaviour is instructive. `sdr-slc-rtld` bridges an RTL-SDR dongle: block rx at Tier 2, no transmitter. `sdr-slc-bladerfd` drives a Nuand bladeRF 2.0 micro: blocks rx and tx, with scheduled-burst transmit for TDMA modes. The macOS client, SDR-SLC Client 1.0, is developed against both. Where this document says "the reference implementation does X", it means one of these two daemons.

## 1.1 What changed in Release 1.3, and why

Release 1.2 introduced function blocks and was, for the first time, implemented on a transceiver rather than only a receiver. Doing so exposed a set of places where the text was written with a receiver in mind and a transmitter had to guess. Every one of those guesses is now written down.

The changes fall into three groups. First, the transmit block: controls on the tx block are now explicit, the client side of `open_stream` for transmit is defined, `set_tx_power` has a parameter table, and a new `get_timestamps` command supports placing transmit bursts on an exact sample — the requirement that TDMA digital voice imposes. Second, the two shipped devices share a sample clock between receive and transmit; a client that does not know this can take down a running receiver by retuning the transmitter, so the sharing is now declared. Third, the companion transport specification SDR-SLC-VITA-1.0 was corrected to be a true VITA-49 subset in its Release 1.2, and the clock rules here are aligned with it.

Nothing in this release changes the receive path of a Tier 1 device. A weekend RTL-SDR bridge built against Release 1.2 remains conformant to Release 1.3 without modification.

## 1.2 Breaking changes

| Change                                                                                                                              | Reason it is not backward compatible                                                                                                                                                                        |
|-------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>turnaround_mode</code> and <code>full_duplex</code> move from the top level of capabilities into <code>capabilities.tx</code> | Release 1.2 §15.1 showed them at the top level while §24.4 required them inside the tx object. Both implementations followed the example. A property of the transmitter belongs in the transmitter's block. |
| <code>set_tx_power</code> is addressed by function, not <code>stream_id</code>                                                      | The command was never given a parameter table. The only implementation addressed it by stream, which is inconsistent with every other configuration command.                                                |
| <code>signal_type</code> restored to the <code>open_stream</code> result                                                            | Removed between the 1.3 draft and Release 1.2 without being listed; every device still sends it and the client reads it.                                                                                    |
| Discovery TXT fw carries the daemon name                                                                                            | Two daemons at the same version were indistinguishable in a discovery list. See §6.                                                                                                                         |

Everything else in this release is additive: new optional fields, one new command that a device may omit, and clarifications of text that implementations had already interpreted the way this release now states.

## 2. Normative Language

The following keywords appear throughout this document and have precise meanings defined by RFC 2119:

| Keyword    | Meaning                                                                                        |
|------------|------------------------------------------------------------------------------------------------|
| SHALL      | Absolute requirement. An implementation that omits or contradicts this is non-conformant.      |
| SHALL NOT  | Absolute prohibition.                                                                          |
| SHOULD     | Strong recommendation. An implementation may deviate but must have a good reason for doing so. |
| SHOULD NOT | Strong recommendation against.                                                                 |
| MAY        | Optional. A conformant implementation may include or omit this feature freely.                 |

Requirement markers appear in command headings. They name the function block and the tier at which the command becomes required:

| Marker                  | Meaning                                                                              |
|-------------------------|--------------------------------------------------------------------------------------|
| [ENVELOPE]              | Required of every SDR-SLC device, regardless of what it does.                        |
| [RX-T1]                 | Required for the rx function block at Tier 1, and at all higher tiers of that block. |
| [RX-T2] / [RX-T3]       | Required for the rx block at Tier 2 or Tier 3.                                       |
| [TX-T1] ... [TX-T3]     | Required for the tx block at the stated tier.                                        |
| [CODEC-T1] / [CODEC-T2] | Required for the codec block at the stated tier.                                     |
| [ANY]                   | May be present on any device. Never required, but always permitted.                  |

|             |                                                                                                                                                                                |
|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>NOTE</b> | A device implements the envelope plus zero or more function blocks. A marker such as [RX-T2] places no obligation whatsoever on a device that does not implement the rx block. |
|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

### 3. Device Model: Envelope and Function Blocks

Every SDR-SLC device consists of an envelope and one or more function blocks.

```

+-- ENVELOPE -- required of every device -----+
|
|  mDNS advertisement (SLC-DISC, section 6)
|  TCP 4620, length-prefixed JSON (section 8)
|  request / response / event (section 9)
|  hello (section 10)
|  get_capabilities (section 11)
|  error code registry (section 22)
|  forward compatibility rules (section 23)
|
|  +-- BLOCK rx ----+  +-- BLOCK tx ----+  +-- BLOCK codec +-+
|  | 7 commands      |  | 4 commands      |  | 3 commands      |
|  | VITA-49         |  | VITA-49         |  | RTP              |
|  +-----+        +  +-----+        +  +-----+
|
+-----+

```

The envelope is two commands: `hello` and `get_capabilities`. Everything else belongs to a function block, and every function block is optional.

### 3.1 Why the envelope is exactly these two commands

hello is required because it fixes the protocol version before anything else happens. Without it there is no way to publish a version 2 without breaking every existing device. On a microcontroller it is one boolean flag and one if.

`get_capabilities` is required because the entire design rests on clients hardcoding nothing. The moment one client is allowed to assume "this device can probably do 2.048 MS/s", the specification is dead. On a microcontroller it may be a static string constant compiled into the firmware.

Everything outside those two is optional, and `unknown_command` is the only thing a device needs to answer for a command it does not implement.

### 3.2 Function blocks defined in this release

| Block | Purpose                                                                                                | Data transport          |
|-------|--------------------------------------------------------------------------------------------------------|-------------------------|
| rx    | Receive. Tune, configure, and stream I/Q samples to a client.                                          | VITA-49 (VITA-A/1.0)    |
| tx    | Transmit. Accept I/Q samples from a client and radiate them, continuously or as scheduled bursts.      | VITA-49 (VITA-A/1.0-TX) |
| codec | Convert audio and framed digital voice between declared formats, in any direction the device supports. | RTP                     |

Future releases may define further blocks. A client that does not recognise a block ignores it; a device that does not implement a block simply does not advertise it. Neither case is an error.

### 3.3 Combining blocks in one device

A device may implement any combination. An RTL-SDR bridge implements rx. A homebrew HF transceiver implements rx and tx. A codec service on a spare Linux box implements codec and nothing else — no frequency, no sample rate, no gain, and fully conformant.

Blocks are also the unit of access control and of authentication. A station can leave its receiver open to the household while requiring a key to transmit, without either decision affecting the other. See §17 and §26.

A radio board that also carries a vocoder chip implements all three, but advertises them as two separate service instances in discovery. See §6.3 for why.

### 3.4 Function identifiers

Each function block a device implements appears in `get_capabilities` under its own key, and is addressed by name in commands.

The reserved function identifiers are rx, tx and codec. A device SHALL NOT use a reserved identifier for a function other than the one defined here. Vendor-defined function blocks SHALL use a prefixed identifier such as `myco.spectrum`.

### 3.5 The vendor prefix rule

The same prefix rule applies to every name a vendor introduces: function blocks (§3.4), controls (§11.3), capability fields, events (§21.1) and commands. A vendor name is `<vendor>.<name>`, where `<vendor>` is a short lowercase identifier the vendor chooses and uses consistently. Unprefixed names are reserved for this specification.

The rule was stated per category in Release 1.2 and one of the reference implementations introduced an unprefixed command and an unprefixed event before noticing. Stating it once, here, removes the gap: if you did not find the name in this document, prefix it.

## 4. Implementation Profiles

Tiers exist so that you know what the minimum useful implementation is. They are a target to build against, not a gate to pass.

A tier belongs to a function block, not to a device. A device may implement one block at Tier 3 and another at Tier 1. In practice you implement one or two blocks, and each block has a Tier 1 that fits in a weekend.

### 4.1 Block rx

| Tier | What it adds                                        | Commands                                                                                                                                                                                                 |
|------|-----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | Samples flow. Nothing else is required.             | <code>set_frequency</code> , <code>set_sample_rate</code> , <code>set_control</code> (rf_gain only), <code>open_stream</code> , <code>start_rx</code> , <code>stop_rx</code> , <code>close_stream</code> |
| 2    | Standard controls, Reference Level, VITA-49 Context | + <code>ping</code> , <code>get_status</code> , <code>get_control</code> , <code>get_clock_status</code>                                                                                                 |

| Tier | What it adds                                   | Commands                                |
|------|------------------------------------------------|-----------------------------------------|
|      | packets, ADC monitoring, the full event model. |                                         |
| 3    | GPS, PTP, hardware monitors.                   | + set_clock, get_location, set_location |

|            |                                                                                                                                                                                                          |
|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>TIP</b> | Envelope (2 commands) plus block rx at Tier 1 (7 commands) is nine commands in total — exactly the weekend project. <code>sdr-slc-rtld</code> began as exactly these nine and grew to Tier 2 afterwards. |
|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 4.2 Block tx

| Tier | What it adds                                                           | Commands                                                                                                                               |
|------|------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| 1    | PTT and I/Q to the DAC. Continuous transmission.                       | get_tx_capabilities, start_tx, stop_tx, plus <code>function:"tx"</code> on set_frequency, set_sample_rate, set_control and open_stream |
| 2    | Output power control. Scheduled bursts with the VITA-A/1.0-TX profile. | + set_tx_power (or <code>rf_gain</code> on the tx block), get_timestamps                                                               |
| 3    | VSWR measurement and antenna tuning.                                   | + get_vswr, auto_tune                                                                                                                  |

Transmit capability does not imply advanced features. A home-built HF transceiver with basic PTT is a legitimate tx Tier 1 device. VSWR monitoring, GPS, PTP clocking, hardware monitors and auto-tuning are Tier 3 concerns. Scheduled bursts are Tier 2 because they need a sample counter the client can read, which not every DAC path has.

## 4.3 Block codec

| Tier | What it adds                                              | Commands                                           |
|------|-----------------------------------------------------------|----------------------------------------------------|
| 1    | One codec instance at a time, one direction, frames flow. | open_stream, start_codec, stop_codec, close_stream |
| 2    | Several concurrent instances, per-instance statistics.    | + get_status                                       |

## 4.4 Declaring a tier

A device SHALL declare, for each function block it implements, the tier at which it implements that block, in `capabilities.<block>.tier`.

Clients SHALL NOT make behavioural decisions based on the declared tier. Clients SHALL determine what a device can do by reading its capabilities.

The second rule matters. A tier is a shorthand for humans and a build target for implementers; capabilities are the truth. A client that switches on the tier will eventually meet a device that implements a Tier 2 feature voluntarily while declaring Tier 1 — `sdr-slc-bladerfd` does exactly this — and will get it wrong.

# 5. Building a Minimal SDR-SLC Device

This chapter is a practical guide for the experimenter who wants samples flowing as quickly as possible. It shows exactly what to implement, in what order, and what to skip.

## 5.1 The complete Tier 1 receive conversation

```
// 1. Client connects on TCP port 4620.
// 2. Client sends hello. Always first.
C->S { "type":"request", "id":"h-1", "command":"hello",
      "params":{"client_name":"SDR-SLC Client", "client_version":"1.0",
               "supported_protocol_versions":["1.0"]} } }
S->C { "type":"response", "id":"h-1", "status":"ok",
      "result":{"protocol_name":"SDR-SLC Control Plane",
               "selected_protocol_version":"1.0",
               "device_name":"My RTL-SDR Bridge",
               "manufacturer":"PA2IX" } } }
```

```

// 3. Client reads capabilities. This may be a static string in firmware.
C->S { "type":"request", "id":"cap-1", "command":"get_capabilities", "params":{} }
S->C { "type":"response", "id":"cap-1", "status":"ok",
  "result":{"capabilities":{"
    "schema":"sdrslc.capabilities", "schema_version":2,
    "rx":{"
      "tier":1,
      "frequency_ranges_hz":[{"min":100000, "max":1766000000 }],
      "sample_rates_sps":[960000, 1024000, 1536000, 2048000],
      "transports":[
        { "protocol":"vita49", "protocol_version":"VITA-A/1.0",
          "source_port":4621, "samples_per_packet":360 }
      ],
      "iq_formats":[
        { "encoding":"unsigned", "bits_per_component":8,
          "container_bits":8, "packing":"interleaved",
          "byte_order":"big_endian",
          "native":true, "supported":true, "label":"u8" }
      ],
      "controls":[
        { "name":"rf_gain", "type":"float", "unit":"dB",
          "min":0, "max":49.6, "step":0.1, "default":20 }
      ]
    }}
  }

// 4. Client tunes to 14.1 MHz.
C->S { "type":"request", "id":"f-1", "command":"set_frequency",
  "params":{"function":"rx", "frequency_hz":14100000 } }
S->C { "type":"response", "id":"f-1", "status":"ok",
  "result":{"function":"rx", "applied_frequency_hz":14100000 } }

// 5. Client sets the sample rate.
C->S { "type":"request", "id":"sr-1", "command":"set_sample_rate",
  "params":{"function":"rx", "sample_rate_sps":2048000 } }
S->C { "type":"response", "id":"sr-1", "status":"ok",
  "result":{"function":"rx", "applied_sample_rate_sps":2048000 } }

// 6. Client sets gain.
C->S { "type":"request", "id":"g-1", "command":"set_control",
  "params":{"function":"rx", "name":"rf_gain", "value":20.0 } }
S->C { "type":"response", "id":"g-1", "status":"ok",
  "result":{"function":"rx", "name":"rf_gain", "applied_value":20.0 } }

// 7. Client opens a VITA-49 stream. Transport particulars live under "vita".
C->S { "type":"request", "id":"os-1", "command":"open_stream",
  "params":{"function":"rx", "protocol":"vita49",
    "vita":{"
      "protocol_version":"VITA-A/1.0",
      "format":{"encoding":"unsigned", "bits_per_component":8,
        "container_bits":8, "packing":"interleaved",
        "byte_order":"big_endian" },
      "destination":{"ip":"192.168.1.20", "port":60000 }
    }}
  } }
S->C { "type":"response", "id":"os-1", "status":"ok",
  "result":{"stream_id":"rx-1", "function":"rx",
    "sample_rate_sps":2048000,
    "signal_type":"complex_iq",
    "vita":{"
      "source":{"ip":"192.168.5.35", "port":4621 },
      "destination":{"ip":"192.168.1.20", "port":60000 },
      "samples_per_packet":360,
      "stream_id":1, "packet_class":"if_data",
      "timestamp_mode":"utc_picoseconds",
      "format":{"encoding":"unsigned", "bits_per_component":8,
        "container_bits":8, "packing":"interleaved",
        "byte_order":"big_endian",
        "native":true, "supported":true, "label":"u8" }
    }}
  } }

// 8. Client starts the stream.
C->S { "type":"request", "id":"rx-1", "command":"start_rx",
  "params":{"stream_id":"rx-1" } }
S->C { "type":"response", "id":"rx-1", "status":"ok",
  "result":{"function":"rx", "streaming":true } }
// Device emits: { "type":"event", "event":"stream_state",

```

```
//  "params":{ "stream_id":"rx-1", "streaming":true, "function":"rx",
//              "reason":"client_request" } }
// --- UDP VITA-49 packets now flow to 192.168.1.20:60000 ---

// 9. Shutdown.
C->S { "type":"request", "id":"rx-2", "command":"stop_rx",
      "params":{ "stream_id":"rx-1" } }
C->S { "type":"request", "id":"cs-1", "command":"close_stream",
      "params":{ "stream_id":"rx-1" } }
// Client closes the TCP connection.
```

**TIP**

If you are building an RTL-SDR bridge daemon, start with exactly these nine commands. Get them working end-to-end before adding anything else. The client will tell you what it needs by returning error messages for features you have not implemented yet.

**NOTE**

The response in step 7 carries two different identifiers that happen to share a name. `result.stream_id` ("rx-1") is the SLC-CP session handle used in `start_rx`, `stop_rx` and `close_stream`. `result.vita.stream_id` (1) is the 32-bit VITA-49 Stream Identifier that appears in every UDP packet header. The first is a string chosen by the device and may repeat across sessions; the second is an integer that SHALL NOT be reused within a TCP session (SDR-SLC-VITA-1.0 §6.2).

## 5.2 What a minimal device may omit

Everything outside the envelope and the Tier 1 commands of the blocks you claim. The client handles absence gracefully as long as you return the right error.

| Feature                                 | What to return when asked                                         |
|-----------------------------------------|-------------------------------------------------------------------|
| ping, get_status, get_clock_status      | unknown_command                                                   |
| set_clock, get_location, set_location   | unknown_command                                                   |
| get_timestamps                          | unknown_command                                                   |
| Controls other than rf_gain             | not_supported (via set_control)                                   |
| Reference Level fields in capabilities  | Simply omit them                                                  |
| VITA-49 Context packets                 | Simply do not send them; the client assumes complex I/Q           |
| Any function block you do not implement | Omit its key from capabilities; return not_supported if addressed |
| Events other than stream_state          | Simply do not emit them                                           |

**NOTE**

`stream_state` is the one event expected even at Tier 1. Everything else can be omitted.

## 5.3 How a minimal device interoperates with a full client

A full-featured client queries capabilities at startup and adapts its display to what the device reports. Connecting such a client to a Tier 1 receiver produces this:

- No preferred\_sample\_rate\_sps in capabilities, so the client uses the first entry in sample\_rates\_sps.
- No reference\_level\_dbm\_at\_preferred\_gain, so the power display reads "uncalibrated" instead of a dBm value.
- ping returns unknown\_command, so the client disables its connection health monitor and stops sending it.
- get\_status returns unknown\_command, so the client disables the ADC level meter and the LO lock indicator.
- set\_control for agc returns not\_supported, so the client greys out the AGC checkbox.
- No Context packets arrive, so the client assumes complex I/Q and proceeds.
- No tx key in capabilities, so transmit controls are hidden.
- No codec key in capabilities, so codec features are hidden. The client does not treat this as an error.

The operator gets a working waterfall and receiver with manual gain control. Adding features later does not require a client update, because the client already knows how to use them when they appear.

## 5.4 Startup sequence

Regardless of what a device implements, the sequence is always the same:

1. TCP connection accepted on port 4620.
2. hello received and validated. Protocol version selected and returned.
3. `get_capabilities` received. Full capabilities returned.
4. `authenticate`, if any function the client needs requires it (§26).
5. Configuration commands, in any order.
6. `open_stream`. Session created, `stream_id` returned.
7. Start command for the relevant block. Streaming begins, `stream_state` emitted.
8. Stream runs. Events emitted as conditions change.
9. Stop command. Streaming stops, `stream_state` emitted.
10. `close_stream`. Session released.
11. TCP connection closed. Device tears down anything still open.

## 6. Discovery

Discovery is specified in full in SDR-SLC-DISC-1.0. This section covers only what Releases 1.2 and 1.3 change, because the changes affect every device.

### 6.1 The TXT field functions replaces role

Release 1.1 advertised `role=rx|tx|rxtx`. That value set cannot express a codec service, and widening it is worse than replacing it: a strict parser rejects an unknown enum value and drops the whole instance, often with an error shown to the operator. A new field is invisible to an old parser; a new value in an existing field is not.

```
_sdr-slc._tcp.local.  
  name=SLC-RP0001  
  proto=sdr-slc  
  proto_ver=1.0  
  functions=rx,tx  
  hw=RP-125-14  
  fw=sdr-slc-bladerfd/1.0  
  minhz=100000  
  maxhz=1766000000
```

The functions TXT value is a comma-separated list of function block identifiers. Clients SHALL ignore identifiers they do not recognise. Clients SHALL NOT reject a service instance because it advertises a function they do not know.

That last sentence is the whole forward compatibility of discovery in one line. Without it, every existing client trips over the first new kind of device — which is exactly what would have happened to `role`.

### 6.2 The fw field names the implementation

The `fw` value SHALL be `<implementation>/<version>`, for example `sdr-slc-rtld/1.0` or `sdr-slc-bladerfd/1.0`. Release 1.2 said only "firmware version", and a discovery list showing two devices both at 1.0 told the operator nothing. A client MAY display the value verbatim; it SHALL NOT parse it for any decision.

The instance name (`name=`) is the operator-facing identity of the device and the key under which clients store it. It SHALL NOT change when the implementation is renamed or upgraded.

### 6.3 A codec service has its own service type

```
_slc-codec._tcp.local.  
  name=NUC-Codec-01  
  proto=sdr-slc  
  proto_ver=1.0  
  functions=codec  
  codecs=codec2,ambe           (hint only, not normative)
```

A program looking for a radio browses `_sdr-slc._tcp` and sees only radios, with nothing to filter. A program that also wants a codec browses a second service type. That is one extra line of code, and it saves every client a filtering step that somebody will forget.

The codecs TXT value is a hint, sufficient to decide whether to connect. It MAY be incomplete. The enumerable list in `get_capabilities` is normative and SHALL be complete.

|             |                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>NOTE</b> | TXT records have a size limit and cannot express which combinations of codec, mode and rate are valid. That is why the hint is deliberately coarse and the authoritative list travels over the control plane. The same pattern already applies to a radio: <code>minhz</code> and <code>maxhz</code> in TXT are enough to decide whether to connect; <code>frequency_ranges_hz</code> in capabilities is the truth. |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 6.4 One device that does both

An STM32 carrying a vocoder chip publishes two independent service instances:

|                              |                             |                              |                          |
|------------------------------|-----------------------------|------------------------------|--------------------------|
| <code>_sdr-slc._tcp</code>   | <code>name=SLC-UHF01</code> | <code>functions=rx,tx</code> |                          |
| <code>_slc-codec._tcp</code> | <code>name=SLC-UHF01</code> | <code>functions=codec</code> | <code>codecs=ambe</code> |

A device implementing both radio and codec functions SHALL advertise them as separate service instances. Clients SHALL NOT infer any relationship between a radio instance and a codec instance that share a name or an address.

Sharing a name and an address is informative. A client may display it ("radio and codec on one device"); it may not decide on it. The test of whether this is right: moving the codec to a different machine should require zero changes in client code.

## 7. Terminology

| Term            | Definition                                                                                                                                                                                                       |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SDR-SLC Client  | Host software that opens an SLC-CP control connection.                                                                                                                                                           |
| SDR-SLC Device  | Any hardware or software that acts as an SLC-CP server. Not necessarily a radio.                                                                                                                                 |
| Envelope        | The commands and rules every device implements: framing, message model, hello, <code>get_capabilities</code> , error codes, forward compatibility.                                                               |
| Function block  | A named, independently claimable set of commands and capability fields. This release defines rx, tx and codec.                                                                                                   |
| Function        | One implemented function block on a device, addressed by its identifier in the <code>function</code> parameter.                                                                                                  |
| Channel         | One independently tunable signal path within the rx or tx block. A device with a single path omits channels entirely.                                                                                            |
| Connector       | One physical port on a signal path that has hardware of its own — a bias tee, a filter, an LNA — selectable with the <code>antenna control</code> . A path with a single port omits connectors entirely (§12.5). |
| Request         | A client-initiated message with a <code>Command</code> field and optional <code>params</code> .                                                                                                                  |
| Response        | A device reply. Carries the <code>id</code> of the request and a <code>status</code> of ok or error.                                                                                                             |
| Event           | An asynchronous device message. Not correlated to any request.                                                                                                                                                   |
| id              | A client-generated string that uniquely identifies a request within the session. Echoed by the device.                                                                                                           |
| Stream ID       | An opaque string assigned by the device to identify an open stream session. Distinct from the VITA-49 numeric Stream Identifier (§5.1 NOTE).                                                                     |
| Codec instance  | One configured codec conversion: one input, one output, one direction. Represented as a stream.                                                                                                                  |
| Reference Level | The power at the antenna port corresponding to a full-scale I/Q sample, in dBm.                                                                                                                                  |
| dBFS            | Decibels relative to full scale. 0 dBFS is the maximum representable sample amplitude.                                                                                                                           |
| Session         | The period from TCP connection to TCP disconnection. All stream state is scoped to a session.                                                                                                                    |
| Turnaround      | The transition from RX to TX or TX to RX in a half-duplex system.                                                                                                                                                |
| Sample counter  | A free-running count of samples converted by the device, per direction. Read with <code>get_timestamps</code> ; used to schedule transmit bursts.                                                                |

| Term  | Definition                                                                                                                                             |
|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| Burst | A bounded transmission with a defined start sample and end, as opposed to continuous transmission. Delimited on the wire by the VITA-A/1.0-TX trailer. |

## 8. Transport and Framing [ENVELOPE]

### 8.1 TCP connection

SLC-CP runs over TCP/IP. The device SHALL listen on TCP port 4620 by default. This port is advertised in the SLC-DISC TXT record, so clients learn it automatically.

A device MAY accept several simultaneous client connections. What happens when two clients want the same resource is specified in §17; it is not implementation-defined.

|            |                                                                                                                                                                         |
|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>TIP</b> | On an STM32 with lwIP, one TCP PCB on port 4620 with a receive callback that dispatches to your command handler is sufficient. You do not need a thread per connection. |
|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

### 8.2 Message encoding

Every message is a single UTF-8 encoded JSON object. Multi-byte Unicode is permitted in string values. Control characters (0x00–0x1F) SHALL NOT appear unescaped inside JSON strings. The device SHALL accept messages up to 65,536 bytes of payload. Longer messages MAY be rejected by closing the connection.

A device SHALL accept both / and \/ for a forward slash inside a string, since both are valid JSON and some serialisers emit the escaped form unasked.

### 8.3 Length-prefixed framing

Each message is framed with a 4-byte big-endian unsigned integer giving the byte length of the JSON payload, followed immediately by the payload:

|           |                                                       |
|-----------|-------------------------------------------------------|
| Frame = [ | length_prefix : uint32, 4 bytes, big-endian ]         |
|           | [ payload : UTF-8 JSON, exactly length_prefix bytes ] |

The length\_prefix covers the payload bytes only, not itself. A frame with length\_prefix=47 is 4 + 47 = 51 bytes on the wire.

|            |                                                                                                                                                                                                                         |
|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>TIP</b> | In C on an STM32 or lwIP: receive 4 bytes, <code>ntohl()</code> to get the payload length, then receive exactly that many more bytes. Pass the payload to your JSON parser. cJSON is small enough to run on an STM32H7. |
|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|             |                                                                                                                                                                                                                                                                                                                              |
|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>NOTE</b> | A device MAY additionally accept newline-delimited JSON on the same port for debugging with nc, detected from the first byte (a length prefix under 16 MiB always begins with 0x00; a JSON object begins with {). Both reference daemons do this and log such a connection as non-conformant. A client SHALL NOT rely on it. |
|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

### 8.4 Field conventions

- All fields ending in `_hz` are 64-bit unsigned integers. On STM32 or 32-bit Linux use `uint64_t`; in JavaScript use `BigInt`. A 32-bit integer only reaches ~4.3 GHz.
- Fields named `epoch_s` are 64-bit signed integers (seconds since the Unix epoch).
- Fields ending in `_sample_count` are 64-bit unsigned integers.

## 9. Message Model [ENVELOPE]

There are three message types: request (client to device), response (device to client), and event (device to client, unsolicited).

## 9.1 Request

```
{
  "type"      : "request",
  "id"        : "any-unique-string-you-choose",
  "command"   : "command_name",
  "params"    : { /* command-specific parameters */ }
}
```

| Field   | Type   | Required | Description                                                                                                 |
|---------|--------|----------|-------------------------------------------------------------------------------------------------------------|
| type    | string | YES      | Literal: "request".                                                                                         |
| id      | string | YES      | Client-chosen identifier, echoed in the response. Unique within the session. A counter or a UUID both work. |
| command | string | YES      | Command name.                                                                                               |
| params  | object | COND     | Command parameters. May be omitted for commands that take none; {} is also acceptable.                      |

## 9.2 Successful response

```
{ "type":"response", "id":"echoed-from-request", "status":"ok",
  "result":{" /* command-specific result fields */ } }
```

## 9.3 Error response

```
{ "type":"response", "id":"echoed-from-request", "status":"error",
  "error":{"
    "code"      : "error_code_string",
    "message"   : "Human-readable explanation for logging or display",
    "details"   : { /* optional structured context */ }
  } }
```

`error.code` is a machine-readable string from the registry in §22. `error.message` is for humans. `error.details` may carry structured context specific to the code.

## 9.4 Asynchronous event

```
{ "type":"event", "event":"event_name",
  "params":{" /* event-specific parameters */ } }
```

Events carry no `id`. They are emitted whenever a device condition occurs, independent of any request. Clients SHALL silently ignore events they do not recognise.

Unknown fields in any message SHALL be silently ignored. This is the fundamental extensibility rule of SLC-CP. If a future client adds a field your device does not understand, your device must still process the message correctly without it.

## 9.5 Response correlation

Clients SHALL correlate responses to requests using the `id` field. Clients SHALL NOT assume that responses arrive in the order the requests were sent, and SHALL NOT assume that the next non-event message is the response to the most recent request.

Getting this wrong produces a failure mode that is hard to recognise: under a burst of commands, a client that reads positionally will silently pair each response with the wrong request, and every value it reads back will be plausible but belong to a different command. The client SHALL also keep a record of every request it has sent until the response arrives, so that a response to a fire-and-forget command (a tune while dragging, a periodic ping) is recognised rather than logged as unexpected.

|            |                                                                                                                                    |
|------------|------------------------------------------------------------------------------------------------------------------------------------|
| <b>TIP</b> | A device may answer in any order it likes, but answering in order is simpler and always allowed. The requirement is on the client. |
|------------|------------------------------------------------------------------------------------------------------------------------------------|

## 10. Version Negotiation [ENVELOPE]

### 10.1 hello

Purpose. Establish the protocol version and exchange device identity. `hello` SHALL be the first command sent by the client after connecting. The device SHALL NOT process any other command until `hello` has completed successfully. If a client sends any command before `hello`, the device SHALL return `invalid_request`.

Request fields

| Field                                    | Type         | Required | Description                                                                                                                                                                                                   |
|------------------------------------------|--------------|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>client_name</code>                 | string       | YES      | Human-readable name of the client application.                                                                                                                                                                |
| <code>client_version</code>              | string       | YES      | Client software version string. Informational only.                                                                                                                                                           |
| <code>supported_protocol_versions</code> | string array | YES      | Protocol versions the client supports. The only protocol version defined by this document is "1.0"; release numbers (1.1, 1.2, 1.3) are document revisions, not protocol versions, and SHALL NOT appear here. |

Response fields

| Field                                  | Type   | Description                                                                                      |
|----------------------------------------|--------|--------------------------------------------------------------------------------------------------|
| <code>protocol_name</code>             | string | Always "SDR-SLC Control Plane".                                                                  |
| <code>selected_protocol_version</code> | string | The version the device selected from the client's list.                                          |
| <code>device_name</code>               | string | Human-readable name of the device.                                                               |
| <code>manufacturer</code>              | string | Manufacturer of the hardware.                                                                    |
| <code>firmware_version</code>          | string | Optional. <code>&lt;implementation&gt;/&lt;version&gt;</code> , matching the discovery fw field. |
| <code>auth</code>                      | object | Present if and only if a function requires authentication. See §26.2.                            |

Behaviour

- The device SHALL select the highest version in the client's list that it also supports.
- If no version is mutually supported, the device SHALL return `unsupported_protocol_version` and MAY close the connection.
- The device MAY include additional informational fields, such as hardware revision or serial number. Clients SHALL ignore fields they do not know.

|             |                                                                                                                                                                                                                                                                                                  |
|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>NOTE</b> | <code>hello</code> deliberately says nothing about what the device is. Device identity beyond a name belongs in <code>get_capabilities</code> , where it can be described properly. This is what makes <code>hello</code> work unchanged for a radio, a codec service, and anything added later. |
|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

```
C->S { "type":"request", "id":"h-1", "command":"hello",
      "params":{"client_name":"SDR-SLC Client", "client_version":"1.0.0",
                "supported_protocol_versions":["1.0"]} }
S->C { "type":"response", "id":"h-1", "status":"ok",
      "result":{"protocol_name":"SDR-SLC Control Plane",
                "selected_protocol_version":"1.0",
                "device_name":"SLC-BLADERF-01",
                "manufacturer":"Nuand",
                "firmware_version":"sdr-slc-bladerfd/1.0" } }
```

|            |                                                                                                                      |
|------------|----------------------------------------------------------------------------------------------------------------------|
| <b>TIP</b> | On an STM32, set a boolean flag after a successful <code>hello</code> . Check it at the start of every other command |
|------------|----------------------------------------------------------------------------------------------------------------------|

|  |                                                                                                   |
|--|---------------------------------------------------------------------------------------------------|
|  | handler; if false, return <code>invalid_request</code> . That is the only session guard you need. |
|--|---------------------------------------------------------------------------------------------------|

## 10.2 ping [ANY • required at RX-T2 and CODEC-T2]

Purpose. Test whether the control connection is alive and measure round-trip latency. No state change, no side effects.

```
C->S { "type":"request", "id":"p-1", "command":"ping", "params":{} }
S->C { "type":"response", "id":"p-1", "status":"ok", "result":{} }
```

The device SHALL respond as quickly as possible, without waiting for any hardware operation. A device that does not implement it returns `unknown_command`; a client that receives that answer SHALL stop sending ping for the rest of the session.

ping belongs to the envelope so that a device with only a tx block has somewhere to put it. It is required at rx Tier 2 and codec Tier 2 and permitted everywhere.

## 11. Capability Exchange [ENVELOPE]

`get_capabilities` is the second command a client calls. It is the foundation of the session: the client reads capabilities and adapts everything else — its controls, its defaults, its knowledge of what is available — to what the device reports.

### 11.1 Structure

```
{ "capabilities": {
  "schema"      : "sdrslc.capabilities",
  "schema_version" : 2,
  "rx"          : { "tier": 1, ... }, // present only if implemented
  "tx"          : { "tier": 1, ... }, // present only if implemented
  "codec"       : { "tier": 1, ... }  // present only if implemented
} }
```

The capabilities object SHALL contain `schema` and `schema_version`, and SHALL contain a key for each function block the device implements.

A device SHALL NOT include a key for a function block it does not implement. Clients SHALL treat the absence of a key as the absence of the function, and SHALL NOT treat it as an error. Clients SHALL ignore function block keys they do not recognise.

### 11.2 Schema versioning

`schema_version` is owned by this specification. Release 1.2 and Release 1.3 both define `schema_version: 2`; Release 1.3 adds only optional fields, which under §23 does not require a new schema number. A device SHALL NOT advertise a `schema_version` that this specification has not defined. Vendor additions go under prefixed keys (§3.5), not under a new schema number.

A client SHALL NOT fail on a `schema_version` higher than the one it knows, provided the protocol version negotiated in `hello` matches. It SHALL process the fields it recognises and ignore the rest.

### 11.3 Control descriptors

Every entry in a `controls[]` array has a type that tells the client what kind of value to send in `set_control`, and which additional fields the descriptor carries.

| type      | Additional fields                                                                | Value type in <code>set_control</code> |
|-----------|----------------------------------------------------------------------------------|----------------------------------------|
| "float"   | min, max, step, unit (optional), default, <code>allowed_values</code> (optional) | JSON number, e.g. 24.0                 |
| "integer" | min, max, step, default                                                          | JSON integer, e.g. -14                 |
| "boolean" | default                                                                          | JSON true or false                     |

| type   | Additional fields              | Value type in set_control                |
|--------|--------------------------------|------------------------------------------|
| "enum" | values (string array), default | JSON string from values, e.g. "i_branch" |

step is optional on a float control that carries `allowed_values`: the list already says which values are legal, and a step would either repeat it or contradict it. A descriptor MAY also carry `connector` when the control belongs to one physical port of the path; see §12.5.

A descriptor describes the control; it does not carry the control's current value. The current value is read with `get_control` or `get_status`.

Standard control names

These names are reserved and have defined meanings. Vendor-specific controls SHALL use a prefixed name such as `myco.custom_filter`.

A reserved control name has the same meaning in whichever function block declares it. `rf_gain` and `bias_tee` in `capabilities.tx.controls` describe the transmit chain's gain and the transmit connector's bias tee. A device SHALL NOT use a reserved name for a different quantity in a different block. The tier column below gives the tier for the rx block; on the tx block every listed control is optional, and a device declares whichever it has.

| name                         | type    | Tier (rx)  | Description                                                                                                                                                                                                                               |
|------------------------------|---------|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>rf_gain</code>         | float   | RX-T1      | Combined RF/IF gain in dB. Every receiver exposes this control. On tx: transmit chain gain.                                                                                                                                               |
| <code>agc</code>             | boolean | RX-T2      | Hardware AGC. When true, the hardware manages <code>rf_gain</code> and <code>set_control</code> for <code>rf_gain</code> is ignored. Receive only.                                                                                        |
| <code>bias_tee</code>        | float   | RX-T2      | Bias tee voltage in volts. 0.0 = off. RTL-SDR V3/V4: <code>allowed_values</code> =[0.0, 5.0]. LNB driver: [0.0, 13.0, 18.0]. bladeRF: [0.0, 5.0] on each of rx and tx. One per connector where the hardware has more than one; see §12.5. |
| <code>antenna</code>         | enum    | RX-T2 · TX | Selects the active connector of the path. <code>values</code> is the list of declared connector ids (§12.5). Absent on hardware that cannot switch, and on any path that declares no <code>connectors</code> [ ].                         |
| <code>ppm_correction</code>  | integer | RX-T2      | Frequency correction in parts per million. Negative shifts the centre frequency down. On a device with one reference oscillator this is device-wide: see below.                                                                           |
| <code>direct_sampling</code> | enum    | RX-T2      | RTL-SDR HF direct sampling. Values "off", "i_branch", "q_branch". Receive only.                                                                                                                                                           |
| <code>lna_gain</code>        | float   | RX-T3      | Separate LNA stage gain where the hardware exposes it independently.                                                                                                                                                                      |
| <code>if_gain</code>         | float   | RX-T3      | IF gain stage, separate from the LNA.                                                                                                                                                                                                     |

Device-wide controls. `ppm_correction` is declared per block like every other control, but on a device with a single reference oscillator it is one physical quantity. Such a device MAY apply a `ppm_correction` written to any block to every block that shares the oscillator, SHALL report the same value from each of them, and SHOULD say so with `"shared_oscillator": true` in the descriptor. A client that sees that flag shows one control, not two.

|            |                                                                                                                                                                                                                        |
|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>TIP</b> | At Tier 1 you need exactly one control: <code>rf_gain</code> as a float. The <code>type</code> field is required even then, because the client uses it to decide what widget to draw and what value type to send back. |
|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 12. Function Addressing

Commands that act on a specific part of a device carry a function parameter naming the function block they address.

The function parameter identifies which function block a command addresses. Its value SHALL be a function identifier present in `capabilities`. A device SHALL return `not_supported` for a function it does not implement, and `missing_parameter` when function is absent on a command that requires it.

## 12.1 Why this replaced direction

Release 1.1 called this field `direction`, with the values `rx` and `tx`. It was already an addressing mechanism — it said which part of the device you were talking to — but it had a name that promised something about signal direction, and a value set of two. As soon as a third kind of function exists, `"direction": "codec"` is unreadable.

The rename touched every command that carries the field and every event that carries it: `set_frequency`, `set_sample_rate`, `set_control`, `get_control`, `open_stream`, `set_tx_power`, `stream_state`, `frequency_changed`, `sample_rate_changed`, `control_changed`, `control_revoked`. A device built against Release 1.1 expects `direction` and will answer `missing_parameter`; there is no compatibility shim.

## 12.2 Direction within a function

Some function blocks have an internal direction, and each names it in its own terms rather than reusing `function`. The `rx` and `tx` blocks use `channel` to select a signal path. The `codec` block names its input and output formats explicitly with `from` and `to`, which makes a separate direction parameter unnecessary — see §16.

## 12.3 Channels

A device may have more than one independent signal path. A receiver board with two front ends is the common case.

When to use channels and when to use separate services. The question that decides it is what the paths physically share. Tuners inside one converter unit share the antenna, the amplification and the ADC: raise the gain for one and it rises for all of them. Two converter units share none of that. The first case is not two independent receivers; the second is.

Functions that can exist independently SHALL be advertised as separate service instances. Signal paths that share hardware SHALL be expressed as channels within one function block.

```
"rx": {
  "tier": 1,
  "coherent": true,
  "transports": [ { "protocol": "vita49", ... } ],
  "channels": [
    { "id": "a",
      "access": "exclusive",
      "frequency_ranges_hz": [ { "min": 100000, "max": 300000000 } ],
      "sample_rates_sps": [ 192000, 384000 ],
      "controls": [ { "name": "rf_gain", ... } ] },
    { "id": "b", ... }
  ]
}

{ "function": "rx", "channel": "a", "frequency_hz": 14100000 }
```

A device that omits `channels` has exactly one channel. Clients SHALL treat an omitted `channel` parameter as addressing that channel. On a single-channel device, `access`, `frequency_ranges_hz`, `sample_rates_sps` and `controls` appear at block level, exactly as in §5.1.

Controls declared inside a channel apply to that channel only. Controls declared at the block level apply to all channels of the block. A device with two or more channels driven by a common clock SHOULD declare `coherent: true`.

`iq_formats` follows the same rule. It MAY be declared inside a channel when the channels have separate converters and therefore separate native formats; declared at block level it applies to all channels. A channel that declares its own `iq_formats` replaces the block-level list for that channel rather than extending it, so a client never has to merge two lists.

### TIP

If you are building a single-receiver device, ignore this section entirely. Omit `channels`, omit the `channel` parameter, and everything behaves exactly as in §5.1.

### NOTE

Multiple independently tuned narrow streams extracted from one wide channel — sometimes called *slices* — are not specified in this release. A client that receives the wide stream can extract them itself at no cost to the device.

## 12.4 Shared sample clock between blocks

Some hardware runs receive and transmit from one sample clock. The AD9361 on the bladeRF is the common case: the two directions can hold different rates, but reprogramming either rate while the other direction is streaming reconfigures the converter and corrupts the running stream.

A block whose sample clock is shared with another block SHALL declare it:

```
"rx": { ..., "shares_sample_clock_with": ["tx"] },
"tx": { ..., "shares_sample_clock_with": ["rx"] }
```

A client that sees the declaration SHALL NOT issue `set_sample_rate` to any block in the group while a stream is active on any other block in the group, and SHOULD negotiate all rates in the group before opening the first stream. A device MAY additionally refuse such a request with `stream_reconfiguration_required`.

This was learned in the field: a client following the Release 1.2 hints correctly set the transmit rate six milliseconds after starting the receive stream and the receiver never recovered. The declaration costs one array; the client rule costs one check.

## 12.5 Connectors

A signal path may end in more than one physical port. A bladeRF 2.0 micro with both receive SMA connectors wired has two ports on one receive path, each with its own bias tee; a client that only knows "the bias tee" cannot say which one it means. A connector is the unit below a channel: a channel is a path that can be tuned, a connector is a port on that path with hardware of its own.

A block or channel MAY declare `connectors[]`, each entry { "id", "label" }. `id` is the identifier used on the wire; `label` is what the operator sees. A control descriptor in the same block or channel MAY carry `connector` naming a declared `id`; the pair (name, connector) is then unique within that block or channel, so two `bias_tee` descriptors can coexist. A device that omits `connectors[]` has exactly one connector per path, and no control in that block or channel carries `connector` — the RTL-SDR case, where nothing changes.

`set_control` and `get_control` carry `connector` for a control that was declared with one, and omit it otherwise. A device SHALL return `invalid_parameter` for a `connector` value that is not declared, and `missing_parameter` when the addressed control was declared with a `connector` and the request omits it. A device SHALL NOT accept a `connector` on a control that was declared without one; ignoring it would make the client think it had addressed a port that does not exist.

The active connector is selected with the reserved antenna control (§11.3), an enum whose values are the declared connector ids. Controls on a connector that is not the selected antenna are settable, so that a client can pre-arm the bias tee on the port it is about to switch to. A device whose hardware ties those controls to the active path returns `not_supported` for such a request and says so in `error.message`; the client then knows to switch first.

```
"rx": {
  "tier": 2,
  "connectors": [
    { "id": "rx1", "label": "RX1 (SMA, bias tee)" },
    { "id": "rx2", "label": "RX2 (SMA, bias tee)" }
  ],
  "controls": [
    { "name": "rf_gain", "type": "float", "unit": "dB",
      "min": -15, "max": 60, "step": 1, "default": 30 },
    { "name": "antenna", "type": "enum", "values": ["rx1", "rx2"], "default": "rx1" },
    { "name": "bias_tee", "type": "float", "unit": "V", "connector": "rx1",
      "allowed_values": [0.0, 5.0], "default": 0.0 },
    { "name": "bias_tee", "type": "float", "unit": "V", "connector": "rx2",
      "allowed_values": [0.0, 5.0], "default": 0.0 }
  ]
}

C->S set_control { "function": "rx", "name": "antenna", "value": "rx2" }
C->S set_control { "function": "rx", "name": "bias_tee", "connector": "rx2", "value": 5.0 }
C->S get_control { "function": "rx", "name": "bias_tee", "connector": "rx1" }
```

The example is how a bladeRF 2.0 micro with both receive ports wired would be declared: two receive connectors, an antenna enum over them, and one `bias_tee` descriptor per connector. `sdr-slc-bladerfd` currently exposes one

receive and one transmit port and therefore omits connectors [ ]; its receive and transmit bias tees are the two `bias_tee` controls on the rx and tx blocks (§15.1), not two connectors. `rf_gain` carries no connector because the gain stage is behind the switch and applies to whichever port is selected.

|             |                                                                                                                                                                                                                                                                                  |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>NOTE</b> | Connectors are not channels. Both bladeRF receive ports feed one tuner and one ADC, so they cannot be tuned independently and are one channel with two connectors. Two front ends with their own converters are two channels (§12.3), each of which may in turn have connectors. |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|            |                                                                                                                                                                                                                              |
|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>TIP</b> | If every path on your device has one port, ignore this section. Omit <code>connectors [ ]</code> , omit <code>connector</code> everywhere, and <code>bias_tee</code> means the only one there is, exactly as in Release 1.2. |
|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 13. Configuration Commands

### 13.1 set\_frequency [RX-T1 • TX-T1]

Purpose. Set the centre frequency of a function. The device tunes as close as the hardware permits and returns the frequency actually applied, which may differ due to PLL step size. Receive and transmit frequencies are independent.

| Field        | Type    | Required | Description                                                                                                                             |
|--------------|---------|----------|-----------------------------------------------------------------------------------------------------------------------------------------|
| function     | string  | YES      | "rx" or "tx".                                                                                                                           |
| channel      | string  | NO       | Channel identifier. Omit on single-channel devices.                                                                                     |
| frequency_hz | integer | YES      | Target centre frequency in Hz. 64-bit unsigned. Must fall within a range in the addressed function's <code>frequency_ranges_hz</code> . |

Result: function, channel (if given), `applied_frequency_hz`.

- The device SHALL apply the frequency immediately. If a stream is active, the change takes effect without interrupting it.
- `applied_frequency_hz` SHALL be what the hardware achieved, not what was requested.
- The device MAY emit a `frequency_changed` event once the synthesiser has settled.

Errors: `unsupported_frequency`, `invalid_parameter`, `missing_parameter`, `not_supported`.

### 13.2 set\_sample\_rate [RX-T1 • TX-T1]

Purpose. Set the I/Q sample rate of a function. Only rates present in the addressed function's `sample_rates_sps` are accepted.

The device SHALL accept only rates from its published `sample_rates_sps` list, and SHALL return `unsupported_sample_rate` for any other value. The discrete list is normative; a device SHALL NOT additionally advertise a minimum/maximum range for sample rates.

- Changing the rate while a stream is active may require the stream to be reopened. If so, return `stream_reconfiguration_required`. A device with a shared sample clock (§12.4) SHALL return it when any block in the clock group is streaming.
- If the achieved rate differs from the requested rate, report the real value in `get_status` as `actual_sample_rate_sps`. `applied_sample_rate_sps` in the response is the nominal rate accepted from the list.

Errors: `unsupported_sample_rate`, `stream_reconfiguration_required`, `not_supported`.

### 13.3 set\_control [RX-T1 • RX-T2 • TX-T1]

Required at RX-T1 for `rf_gain`, RX-T2 for the other standard controls, and TX-T1 for any control the tx block declares.

Purpose. Set a named hardware control. The available controls are reported in the addressed function's (or channel's) `controls [ ]`. The value type follows the descriptor's type.

| Field     | Type                   | Required | Description                                                                                          |
|-----------|------------------------|----------|------------------------------------------------------------------------------------------------------|
| function  | string                 | YES      | Function identifier.                                                                                 |
| channel   | string                 | NO       | Required only if the control is declared inside a channel.                                           |
| name      | string                 | YES      | Control name as listed in <code>controls[]</code> .                                                  |
| connector | string                 | NO       | Required only if the control's descriptor carries <code>connector</code> (§12.5); omitted otherwise. |
| value     | number / bool / string | YES      | New value, matching the control's declared type.                                                     |

Result: `function`, `name`, `applied_value` — the value actually applied, which may be quantised to the nearest step — and `connector` when the request carried it.

#### Behaviour

- When a gain control changes on the rx block, the device SHALL [RX-T2] emit a VITA-49 Context packet with the updated Reference Level before the next IF Data packet.
- When `agc` is set true, the device ignores subsequent `rf_gain` commands until it is set false. It SHOULD emit `control_changed` events as the AGC adjusts gain, and SHALL emit Context packets on each AGC gain change.
- When `agc` returns to false, the device SHALL restore the last client-set `rf_gain`.
- `bias_tee` SHALL default to 0.0 V at startup and after any TCP disconnect, on every connector. On the tx block the ordering rule of §15.7 applies.
- When `direct_sampling` changes, the device SHALL emit a Context packet with the updated Payload Format before the next IF Data packet.
- For float controls with `allowed_values`, the value SHALL match one of the listed values exactly; otherwise return `invalid_parameter`.

Errors: `not_supported`, `invalid_parameter` (including an undeclared connector), `missing_parameter` (including a connector omitted on a control that requires one).

```
// float
C->S set_control { "function":"rx", "name":"rf_gain", "value":24.0 }
// boolean
C->S set_control { "function":"rx", "name":"agc", "value":true }
// float with allowed_values
C->S set_control { "function":"rx", "name":"bias_tee", "value":5.0 }
// the same control on the transmit connector
C->S set_control { "function":"tx", "name":"bias_tee", "value":5.0 }
// a control declared with a connector (§12.5)
C->S set_control { "function":"rx", "name":"bias_tee", "connector":"rx2", "value":5.0 }
// integer
C->S set_control { "function":"rx", "name":"ppm_correction", "value":-14 }
// enum
C->S set_control { "function":"rx", "name":"direct_sampling", "value":"i_branch" }
```

## 13.4 get\_control [RX-T2]

Purpose. Query the current applied value of one named control. Useful after a reconnect, or to verify that a `set_control` took effect.

| Field     | Type   | Required | Description                                                                                          |
|-----------|--------|----------|------------------------------------------------------------------------------------------------------|
| function  | string | YES      | Function identifier.                                                                                 |
| channel   | string | NO       | Required only if the control is declared inside a channel.                                           |
| name      | string | YES      | Control name as listed in <code>controls[]</code> .                                                  |
| connector | string | NO       | Required only if the control's descriptor carries <code>connector</code> (§12.5); omitted otherwise. |

Result: `function`, `name`, `value`, and `connector` when the request carried it. Errors are as for `set_control`.

## 14. Stream Session Management

A stream session represents a configured or active data flow. Every stream has the same lifecycle regardless of which function block it belongs to:

```

open_stream ---> OPENED ---> start_* ---> ACTIVE
                                |
close_stream <--- OPENED <--- stop_* <-----+
TCP disconnect: the device closes all sessions in any state.

```

On TCP disconnect the device SHALL immediately stop all active streams and release all open stream sessions. Stream IDs SHALL NOT be reused within the same TCP session.

Reusing the lifecycle across blocks is deliberate. A codec instance is a stream: one configured conversion with one input, one output, and a start and stop. There is no reason to invent a second set of verbs for it, and every reason not to — the teardown-on-disconnect rule above is exactly what a codec service needs, and it comes for free.

### 14.1 open\_stream [RX-T1 • TX-T1 • CODEC-T1]

Purpose. Configure a data flow for a function. The parameters are split into a common part and a transport-specific part.

| Field      | Type   | Required | Description                                                                                                  |
|------------|--------|----------|--------------------------------------------------------------------------------------------------------------|
| function   | string | YES      | Function identifier: "rx", "tx", "codec".                                                                    |
| channel    | string | NO       | Channel identifier, for multi-channel functions.                                                             |
| protocol   | string | YES      | Transport protocol name from the function's transports[]. "vita49" or "rtp".                                 |
| <protocol> | object | YES      | An object named after the transport, carrying all transport-specific parameters. See §14.2, §14.5 and §16.2. |

The result always carries stream\_id and function, plus an object named after the transport with the confirmed parameters. For the rx and tx blocks it also carries sample\_rate\_sps (the nominal rate in force) and signal\_type.

### 14.2 Why transport parameters are grouped

With more than one transport in the protocol, top-level transport fields become ambiguous. open\_stream SHALL carry transport-specific parameters inside an object named after the transport protocol. Transport-specific parameters SHALL NOT appear at the top level of the request or the response.

### 14.3 Advertising transports

Each function block advertises the transports it can carry data over.

```

{
  "rx": {
    "transports": [
      { "protocol": "vita49", "protocol_version": "VITA-A/1.0",
        "source_port": 4621, "samples_per_packet": 360 }
    ]
  },
  "tx": {
    "transports": [
      { "protocol": "vita49", "protocol_version": "VITA-A/1.0-TX",
        "listen_port": 4622, "samples_per_packet": 360 }
    ]
  },
  "codec": {
    "transports": [
      { "protocol": "rtp", "protocol_version": "1.0" }
    ]
  }
}

```

A device SHALL advertise a transports[] array for every function block that carries data. A device SHALL return unsupported\_stream\_protocol when a client requests a protocol not present in that array.

For the tx block, `protocol_version` names the highest transmit profile the device accepts. A device advertising VITA-A/1.0-TX SHALL also accept plain VITA-A/1.0 packets on the same port as continuous transmission (SDR-SLC-VITA-1.0 §7A); a device advertising VITA-A/1.0 accepts only that.

## 14.4 Ports: who binds what

This deserves its own section because it has already caused a real interoperability failure. A device assumed the client would receive on a fixed port; the client had bound an ephemeral one. Both sides were internally consistent and nothing worked.

`source_port` in `transports[]` is the UDP port from which the device sends. It is never a destination. The destination is always chosen by the client and carried in the transport object of `open_stream`. A device SHALL send to that address and port, and SHALL NOT assume a fixed or default client port.

For transmit the roles reverse: `listen_port` is the UDP port on which the device receives. The client sends to it. The naming rule is the same in both directions: the field says what the device does with the port, so there is nothing left to assume.

## 14.5 The vita object for a transmit stream

For function: "tx" the client is the sender, so it has no destination to give and the device has no source to report. The vita object carries:

| Field                           | Direction | Required | Description                                                                                                                             |
|---------------------------------|-----------|----------|-----------------------------------------------------------------------------------------------------------------------------------------|
| <code>protocol_version</code>   | C→S       | YES      | "VITA-A/1.0" for continuous transmission, "VITA-A/1.0-TX" for scheduled bursts.                                                         |
| <code>format</code>             | C→S       | YES      | I/Q format object, from <code>capabilities.tx.iq_formats</code> .                                                                       |
| <code>source</code>             | C→S       | NO       | { "ip", "port" } from which the client will send. A device SHOULD accept transmit data only from this address when it is given (§25.2). |
| <code>listen_port</code>        | S→C       | YES      | UDP port on the device to send to.                                                                                                      |
| <code>stream_id</code>          | S→C       | YES      | The 32-bit VITA-49 Stream Identifier the client SHALL put in every packet.                                                              |
| <code>samples_per_packet</code> | S→C       | YES      | Advisory for transmit: the device reads the pair count from each packet's size field, and any count that fits the MTU is accepted.      |
| <code>format</code>             | S→C       | YES      | The format the device will decode.                                                                                                      |

```

C->S { "type":"request", "id":"ot-1", "command":"open_stream",
      "params":{"function":"tx", "protocol":"vita49",
        "vita":{"
          "protocol_version":"VITA-A/1.0-TX",
          "format":{"encoding":"twos_complement", "bits_per_component":16,
            "container_bits":16, "packing":"interleaved",
            "byte_order":"big_endian" },
          "source":{"ip":"192.168.1.20", "port":50701 }
        } } }
S->C { "type":"response", "id":"ot-1", "status":"ok",
      "result":{"stream_id":"tx-1", "function":"tx",
        "sample_rate_sps":1024000,
        "signal_type":"complex_iq",
        "vita":{"
          "listen_port":4622, "stream_id":2,
          "samples_per_packet":360,
          "format":{"encoding":"twos_complement", "bits_per_component":16,
            "container_bits":16, "packing":"interleaved",
            "byte_order":"big_endian", "native":true,
            "supported":true, "label":"int16" }
        } } }

```

## 14.6 signal\_type

The `open_stream` result for rx and tx carries `signal_type`: "complex\_iq" or "real". It is the state at the moment the stream was opened and is informational. Where a VITA-49 Context packet is available it is authoritative

and supersedes this field (SDR-SLC-VITA-1.0 §7.7); a Tier 1 device that sends no Context packets is describing its stream here and nowhere else, which is why the field exists.

## 14.7 start\_rx / stop\_rx [RX-T1]

start\_rx begins sending VITA-49 packets to the destination configured in open\_stream. stop\_rx stops them; the session stays open and can be restarted without reopening. Both take stream\_id and return function and streaming.

- The device SHALL emit a stream\_state event with the new streaming value.
- [RX-T2] The device SHALL emit a VITA-A/1.0-CTX Context packet as the first UDP packet after start\_rx, before any IF Data packets.

Errors: stream\_not\_found, stream\_not\_open.

## 14.8 close\_stream [RX-T1 • TX-T1 • CODEC-T1]

Releases a stream session. If the stream is still active, the device SHALL stop it first, then close it. After close\_stream the stream\_id is invalid, and any access control held through it is released (§17). Result: closed: true. Errors: stream\_not\_found.

# 15. Function Block: tx

The transmit block adds four commands to the shared configuration and stream commands. For transmit the data relationship is reversed: the client sends VITA-49 packets to the device.

## 15.1 Capabilities

```

"tx": {
  "tier": 2,
  "access": "exclusive",
  "auth": "psk-hmac-sha256",
  "full_duplex": false,
  "turnaround_mode": "manual", // or "auto"
  "frequency_ranges_hz": [{ "min": 47000000, "max": 6000000000 }],
  "sample_rates_sps": [520834, 1024000, 2048000],
  "preferred_sample_rate_sps": 1024000,
  "preferred_gain_db": 0,
  "shares_sample_clock_with": ["rx"],
  "tx_data_timeout_ms": 250,
  "transports": [
    { "protocol": "vita49", "protocol_version": "VITA-A/1.0-TX",
      "listen_port": 4622, "samples_per_packet": 360 }
  ],
  "iq_formats": [
    { "encoding": "twos_complement", "bits_per_component": 16,
      "container_bits": 16, "packing": "interleaved",
      "byte_order": "big_endian",
      "native": true, "supported": true, "label": "int16" }
  ],
  "controls": [
    { "name": "rf_gain", "type": "float", "unit": "dB",
      "min": -24, "max": 66, "step": 1, "default": 0 },
    { "name": "bias_tee", "type": "float", "unit": "V",
      "allowed_values": [0.0, 5.0], "default": 0.0 }
  ],
  "power_min_dbm": -20, "power_max_dbm": 8, "power_step_dbm": 1
}

```

full\_duplex and turnaround\_mode are properties of the transmitter in relation to the receiver on the same device, and live inside the tx object. Release 1.2's example showed them at the top level; that was an error and it was copied. turnaround\_mode MAY be omitted when full\_duplex is true, because a device that never turns around has no mode to declare; a client SHALL treat the omission on a full-duplex device as "none" and SHALL NOT sequence a turnaround.

A tx block whose path has more than one physical port declares connectors[] and per-connector controls exactly as rx does (§12.5).

`controls[]` on the tx block works exactly as on rx (§11.3, §13.3). A transmitter without controls omits the array. A device that has a transmit-side amplifier powered through a bias tee declares `bias_tee` here, and a client that renders capabilities.`rx.controls` SHALL render capabilities.`tx.controls` the same way under its transmit section.

## 15.2 get\_tx\_capabilities [TX-T1]

Returns transmit capabilities in more detail than the capabilities sub-object. A minimal transceiver may set `vswr_monitor: false`, `auto_tune: false` and omit the power fields entirely.

| Field                                                               | Type    | Description                                                                |
|---------------------------------------------------------------------|---------|----------------------------------------------------------------------------|
| <code>supported</code>                                              | boolean | true if transmit is available.                                             |
| <code>frequency_ranges_hz</code>                                    | array   | Transmit frequency ranges.                                                 |
| <code>sample_rates_sps</code>                                       | array   | Transmit sample rates.                                                     |
| <code>iq_formats</code>                                             | array   | Transmit I/Q formats.                                                      |
| <code>transports</code>                                             | array   | Transport descriptors, including <code>listen_port</code> .                |
| <code>tx_data_timeout_ms</code>                                     | integer | The §15.7 timeout.                                                         |
| <code>vswr_monitor</code>                                           | boolean | [TX-T3] true if <code>get_vswr</code> is available.                        |
| <code>auto_tune</code>                                              | boolean | [TX-T3] true if <code>auto_tune</code> is available.                       |
| <code>power_min_dbm /<br/>power_max_dbm /<br/>power_step_dbm</code> | number  | [TX-T2] Power range. Omit if <code>set_tx_power</code> is not implemented. |
| <code>timestamps</code>                                             | boolean | [TX-T2] true if <code>get_timestamps</code> is available.                  |

## 15.3 start\_tx / stop\_tx [TX-T1]

`start_tx` arms the transmitter: the device begins accepting VITA-49 packets from the client and outputting them to the DAC and PA. If the client sends no packets, the RF output is silence. `stop_tx` stops output and turns off the PA; the session stays open.

- The device emits `stream_state` with the new value and function: "tx".
- If `turnaround_mode` is auto, the device also mutes or resumes receive and emits `stream_state` for the rx function.

## 15.4 PTT and turnaround

Half-duplex transceivers must manage the receive-to-transmit transition. `turnaround_mode` tells the client which side is responsible.

```
// manual turnaround: the client sequences it. Recommended for
// simple homebrew hardware, because the device needs no state machine.
C->S stop_rx { "stream_id":"rx-1" }
C->S start_tx { "stream_id":"tx-1" }
// ... client sends VITA-49 UDP to the device listen port ...
C->S stop_tx { "stream_id":"tx-1" }
C->S start_rx { "stream_id":"rx-1" }

// auto turnaround: the device has hardware T/R switching.
C->S start_tx { "stream_id":"tx-1" }
S->C ok + event stream_state(tx-1, streaming=true, reason=client_request)
S->C event stream_state(rx-1, streaming=false, reason=turnaround)
```

A full-duplex device (`full_duplex: true`) keeps receiving during transmit and emits nothing for rx.

## 15.5 set\_tx\_power [TX-T2]

Purpose. Set the PA output power. It does not start transmission; it configures the level used on the next `start_tx`, and applies immediately if transmission is active.

| Field     | Type   | Required | Description                                                           |
|-----------|--------|----------|-----------------------------------------------------------------------|
| function  | string | YES      | "tx".                                                                 |
| channel   | string | NO       | Channel identifier on multi-channel transmitters.                     |
| power_dbm | number | YES      | Requested output power in dBm, within power_min_dbm... power_max_dbm. |

Result: function, applied\_power\_dbm — quantised to power\_step\_dbm.

```
C->S { "type":"request", "id":"pw-1", "command":"set_tx_power",
      "params":{"function":"tx", "power_dbm":5 } }
S->C { "type":"response", "id":"pw-1", "status":"ok",
      "result":{"function":"tx", "applied_power_dbm":5 } }
```

Power is a property of the transmitter, not of a stream session, which is why it is addressed like every other configuration command rather than by stream\_id. A device that has rf\_gain on the tx block and no calibrated power scale omits set\_tx\_power and the power fields; gain is then the control, and it satisfies the tx Tier 2 requirement in place of set\_tx\_power (§24.5). A tx Tier 2 device SHALL offer at least one of the two.

Errors: unsupported\_power, not\_supported.

## 15.6 get\_vswr and auto\_tune [TX-T3]

get\_vswr returns vswr, forward\_power\_w, reflected\_power\_w, return\_loss\_db, measurement\_frequency\_hz and valid. It requires a directional coupler; return not\_supported if none is fitted.

auto\_tune triggers a tuner cycle at a given frequency\_hz and power\_dbm, returning tuned, vswr\_before, vswr\_after and tune\_duration\_ms. The device SHALL NOT initiate a tune cycle automatically on transmit start or frequency change. It is an operator-initiated command only.

|                |                                                                                                 |
|----------------|-------------------------------------------------------------------------------------------------|
| <b>WARNING</b> | During tuning the device transmits a carrier. The operator must ensure an antenna is connected. |
|----------------|-------------------------------------------------------------------------------------------------|

## 15.7 Transmit safety

A transmitter that keeps radiating because a control connection went away is a hazard independent of any question of access control.

A device implementing the tx block SHALL stop transmitting on loss of the control connection.

A device implementing the tx block SHALL stop transmitting when no transmit data has arrived for longer than a device-defined timeout, and SHALL publish that timeout as tx\_data\_timeout\_ms in capabilities.tx. Both reference daemons use 250 ms.

A device SHALL NOT resume transmitting after either condition without a new start\_tx. When it stops for either reason it SHALL emit stream\_state with streaming:false and reason set to control\_lost or tx\_data\_timeout (§21.2), so that a client whose connection survived knows why the PA dropped.

Where a device powers an external amplifier through a bias tee, it SHALL stop transmit output before removing bias tee power, and SHALL restore bias tee power before resuming transmit output. A device SHALL NOT rely on the client to sequence this. Removing the supply from an amplifier that is still being driven is the case that damages hardware.

|            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>TIP</b> | A few hundred milliseconds is a sensible timeout: long enough to ride out normal network jitter, short enough that a crashed client does not hold the PA on. On a microcontroller this is one timestamp and one comparison in the packet handler. For scheduled bursts (§15.8) the timeout counts from the last packet received, not from the last sample radiated, so a client may pre-load a burst and go quiet until the next one as long as the gaps are shorter than the timeout. |
|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 15.8 get\_timestamps [TX-T2]

Purpose. Read the device's free-running sample counters so the client can schedule a transmit burst on an exact sample. This is what makes TDMA digital voice possible over a network: a DMR burst has to place 27.5 ms of energy inside a 30 ms slot, and the only clock exact enough to do that is the device's own.

```
C->S { "type":"request", "id":"ts-1", "command":"get_timestamps", "params":{} }
S->C { "type":"response", "id":"ts-1", "status":"ok",
  "result":{
    "rx_sample_count"      : 91234567,
    "tx_sample_count"      : 91230000,
    "rx_sample_rate_sps"   : 2048000,
    "tx_sample_rate_sps"   : 1024000,
    "shares_clock"         : true,
    "capture_spread_ns"    : 41000,
    "rx_stream_epoch_sample" : 4,
    "min_burst_lead_us"    : 5000,
    "max_burst_samples"    : 1048576
  } }
```

| Field                                      | Type    | Description                                                                                                                                                                           |
|--------------------------------------------|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| rx_sample_count                            | integer | Device receive sample counter at the moment of the read. Null if the device has no receiver.                                                                                          |
| tx_sample_count                            | integer | Device transmit sample counter at the moment of the read. This is the domain of the VITA-A/1.0-TX timestamp.                                                                          |
| rx_sample_rate_sps /<br>tx_sample_rate_sps | number  | Nominal rates the counters advance at.                                                                                                                                                |
| shares_clock                               | boolean | true if the two counters advance from one clock, so their ratio is exact.                                                                                                             |
| capture_spread_ns                          | integer | Host-time window within which the two counters were read. They cannot be read in one bus transaction; this bounds the conversion error and SHOULD be checked, not assumed small.      |
| rx_stream_epoch_sample                     | integer | Value of the rx counter at the first sample of the currently open receive stream. Adding the elapsed sample count derived from a received VITA timestamp gives an absolute rx sample. |
| min_burst_lead_us                          | integer | Minimum time between the device receiving a burst-start packet and its scheduled sample; earlier arrival is required.                                                                 |
| max_burst_samples                          | integer | Longest burst the device will buffer.                                                                                                                                                 |

The client converts a received sample index into a transmit schedule with  $\text{tx\_sample} = \text{tx\_sample\_count} + (\text{rx\_sample\_target} - \text{rx\_sample\_count}) \times \text{tx\_rate} \div \text{rx\_rate}$ , valid when `shares_clock` is true. When it is false the two counters drift and the client SHALL re-read before every burst.

If the device resets its rx counter underneath an open stream it SHALL emit `rx_epoch_changed` with the new `rx_stream_epoch_sample`, and a client holding a cached mapping SHALL invalidate it. A burst whose scheduled sample has already passed when it arrives is dropped, not transmitted late, and the device emits `tx_burst_late` (§21.2).

The wire format of a scheduled burst is SDR-SLC-VITA-1.0 §7A (VITA-A/1.0-TX). A device implementing this command SHALL advertise `protocol_version: "VITA-A/1.0-TX"` in `capabilities.tx.transports[]`.

Errors: `not_supported` on a device without sample counters; `unknown_command` at Tier 1.

## 16. Function Block: codec

A codec service converts audio and framed digital voice between formats. It is discovered with mDNS and controlled with SLC-CP, exactly like a radio. It uses RTP rather than VITA-49 to carry data, because the payload is short framed packets on a fixed cadence rather than a continuous sample stream.

The block exists because the digital voice modes in general use rely on patent-encumbered vocoders that cannot be embedded in every client. Separating the vocoder into its own network service lets a client detect, display and decode everything it lawfully can, and delegate the rest to whatever hardware or software the operator has. The protocol

itself is codec-agnostic: it describes conversions between named formats, and says nothing about how a device performs them.

## 16.1 Formats and conversions

A codec service declares two things: the formats it understands, and the conversions it can perform between them. PCM is one of the formats, not a fixed side of every conversion.

```

"codec": {
  "tier": 1,
  "access": "shared",
  "transports": [ { "protocol": "rtp", "protocol_version": "1.0" } ],
  "formats": [
    { "id": "pcm8k", "kind": "pcm", "sample_rate_sps": 8000, "format": "s16le", "channels": 1 },
    { "id": "ambe-dmr", "kind": "frame", "codec": "ambe", "mode": "dmr",
      "rate_bps": 2450, "frame_ms": 20, "frame_bits": 49, "frame_bytes": 7 },
    { "id": "c2-700d", "kind": "frame", "codec": "codec2", "mode": "700d",
      "rate_bps": 700, "frame_ms": 40, "frame_bits": 28, "frame_bytes": 4 }
  ],
  "conversions": [
    { "from": "ambe-dmr", "to": "pcm8k" },
    { "from": "pcm8k", "to": "ambe-dmr" },
    { "from": "ambe-dmr", "to": "c2-700d" },
    { "from": "pcm8k", "to": "c2-700d" },
    { "from": "c2-700d", "to": "pcm8k" }
  ]
}

```

A device SHALL declare every format it understands in `formats[]`, each with a unique `id` within the device, and SHALL declare every conversion it can perform in `conversions[]`, referring to those identifiers. A client SHALL NOT request a conversion that is not present in `conversions[]`. A device SHALL return `unsupported_conversion` if one is requested. Clients SHALL ignore formats whose `kind`, `codec` or `mode` they do not recognise, and SHALL ignore conversions that refer to them.

Format descriptors

| Field                                                                      | Applies to | Description                                                                                                                                                                                 |
|----------------------------------------------------------------------------|------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>id</code>                                                            | all        | Identifier, unique within the device. Referenced by <code>conversions[]</code> and by <code>open_stream</code> .                                                                            |
| <code>kind</code>                                                          | all        | "pcm" for uncompressed audio, "frame" for a framed codec payload.                                                                                                                           |
| <code>sample_rate_sps</code> , <code>format</code> , <code>channels</code> | pcm        | Audio sample rate, sample format, channel count.                                                                                                                                            |
| <code>codec</code> , <code>mode</code>                                     | frame      | Codec family and mode, e.g. ambe / dmr.                                                                                                                                                     |
| <code>rate_bps</code> , <code>frame_ms</code>                              | frame      | Bit rate and frame duration.                                                                                                                                                                |
| <code>frame_bits</code>                                                    | frame      | Voice bits per frame. SHALL equal $\text{rate\_bps} \times \text{frame\_ms} \div 1000$ .                                                                                                    |
| <code>frame_bytes</code>                                                   | frame      | Octets per frame on the wire: $\text{ceil}(\text{frame\_bits} \div 8)$ , packed MSB-first, low-order bits of the last octet zero (SDR-SLC-RTP-1.0 §7.1).                                    |
| <code>fec</code>                                                           | frame      | Optional. Present only when the frame carries channel coding in addition to voice bits, naming it, e.g. "dstar-golay". Absent means voice bits only, which is the SDR-SLC-RTP-1.0 boundary. |

A framed descriptor whose `frame_bits` disagrees with its `rate_bps` and `frame_ms` is malformed. Clients SHALL ignore it rather than guess which figure is right; a wrong bit count fails as noise, not as an error. A service that carries protected frames advertises them as a separate format with its own `id` and an explicit `fec`.

|            |                                                                                                                                                                                                                             |
|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>TIP</b> | Identifiers are opaque to the client: it matches on the descriptive fields, not on the string. Use whatever names are convenient, but keep them stable across firmware versions so that stored configurations keep working. |
|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 16.2 Opening a conversion

A codec instance is one configured conversion: one input, one output, one direction. It is represented as a stream and follows the lifecycle in §14.

```
// Decode: AMBE frames in, PCM audio out.
C->S { "type":"request", "id":"oc-1", "command":"open_stream",
  "params":{"
    "function":"codec", "protocol":"rtp",
    "from":"ambe-dmr", "to":"pcm8k",
    "rtp":{"
      "input" : { "frames_per_packet":1 },
      "output": { "destination":{" ip":"192.168.1.20", "port":50000 },
        "frames_per_packet":1 }
    } } }
S->C { "type":"response", "id":"oc-1", "status":"ok",
  "result":{"
    "stream_id":"cv-1", "function":"codec",
    "from":"ambe-dmr", "to":"pcm8k",
    "rtp":{"
      "input" : { "receive":{" ip":"192.168.5.40", "port":50104 },
        "payload_type":97, "clock_rate":8000, "frames_per_packet":1 },
      "output": { "destination":{" ip":"192.168.1.20", "port":50000 },
        "payload_type":98, "clock_rate":8000, "frames_per_packet":1 }
    } } }
C->S start_codec { "stream_id":"cv-1" }
// --- someone sends RTP to 192.168.5.40:50104 (AMBE frames)
// --- device sends RTP to 192.168.1.20:50000 (PCM audio)
C->S stop_codec { "stream_id":"cv-1" }
C->S close_stream { "stream_id":"cv-1" }
```

Encoding is the same exchange with the format identifiers swapped. Mode-to-mode transcoding is the same exchange with two frame formats. There is no separate command and no operation parameter: from and to say everything.

## 16.3 RTP endpoints and port binding

A conversion has two RTP flows in opposite directions, so there are two listening ports and therefore two opportunities to assume that the other end listens somewhere fixed.

The device SHALL bind its own RTP receive port for the input side and SHALL return it in `rtp.input.receive`. The address returned SHALL be a unicast address of the device that is reachable from the client — never the unspecified address `0.0.0.0` or `::`, which a device bound to all interfaces is tempted to report and which sends the client's frames nowhere. The device SHALL NOT assume anything about the address that will send to it.

The client SHALL specify the output destination in `rtp.output.destination`. The device SHALL send output there, and SHALL NOT assume that address belongs to the controlling client. Neither endpoint is named after the client, deliberately: the destination of a conversion is frequently not the client that set it up.

### NOTE

A device MAY additionally restrict its input port to accept media only from an expected source address, and SHOULD do so when the client supplies one in `rtp.input.expect_source`. This is a hardening measure, not a security mechanism; see §25.

## 16.4 Chaining conversions

A conversion the device cannot perform in one step can be built from two, by pointing the output of one instance at the input of another. The network is the pipeline; the service contains no pipeline machinery and is not intended to grow any.

```
// Goal: YSF frames from service A, delivered as DMR frames to a client,
// where neither service can do the conversion in one step.
// 1. Open the SECOND stage first, to learn where it listens.
C->B open_stream { "function":"codec", "protocol":"rtp",
  "from":"pcm8k", "to":"ambe-dmr",
  "rtp":{" output":{" destination":{" ip":"192.168.1.20", "port":50000 } } } }
B->C ok rtp.input.receive = { "ip":"192.168.5.41", "port":50210 }
// 2. Open the FIRST stage, aimed at the second.
C->A open_stream { "function":"codec", "protocol":"rtp",
  "from":"ambe-ysf", "to":"pcm8k",
```

```

        "rtp":{ "output":{ "destination":{ "ip":"192.168.5.41", "port":50210 } } } }
A->C ok rtp.input.receive = { "ip":"192.168.5.40", "port":50104 }
// 3. Start both, second stage first.
C->B start_codec ; C->A start_codec

```

When chaining, a client SHALL open the downstream instance before the upstream one, because the upstream `output.destination` is the downstream `input.receive`.

Chaining is not free: an intermediate PCM stage crosses the network at roughly fifty times the bit rate of the framed formats around it, and adds a network hop of latency. A service that holds both formats should be asked to do the conversion in one step, which is exactly what the `conversions[]` list is for.

## 16.5 Interoperability profile for existing vocoder servers [ANY]

A large installed base of AMBE hardware speaks a UDP protocol that tunnels the vocoder chip's serial framing directly. A device may support it as an additional transport:

```

"transports": [
  { "protocol":"rtp", "protocol_version":"1.0" },
  { "protocol":"dv3000", "protocol_version":"1.0", "listen_port":2460 }
]

```

RTP is the normative transport for the codec block. Any other transport is an optional interoperability profile, and SHALL NOT be the only transport a Tier 1 codec device advertises.

## 16.6 Why JSON negotiation and not SDP

RTP itself is a twelve-byte header and can be implemented in an hour. SDP means generating a media description document and implementing an offer/answer state machine, in order to say "AMBE, DMR mode, 20 ms per frame, send it to port 50000". An SDP blob carried as a string inside a JSON field is an exception to the principle that the whole protocol can be debugged with netcat, and it buys interoperability that does not exist: deployed AMBE-over-network equipment speaks a serial tunnelling protocol, not RTP with SDP. So the transport is RTP and the negotiation is JSON, carrying the same information SDP would have carried.

## 16.7 What the codec block does not cover

- Frame detection and display. Recognising a digital voice transmission, its mode and its metadata is signal processing performed on the received I/Q. It requires no codec service.
- Which formats a device implements. Whether a device performs a conversion in software, on a vocoder chip, or not at all is outside the specification — as is whether the operator holds the licences to do so.
- Routing and mixing. A conversion has one input and one output. Anything with more is built by connecting instances.

# 17. Access Control and Ownership

Release 1.1 stated that behaviour on conflicting commands from several clients was implementation-defined. That is not tenable once devices serve several clients, which is the normal case for a codec service and a realistic case for a multi-channel receiver.

## 17.1 The distinction that drives the model

- A receive channel has one tuned frequency. Two clients tuning it fight over one physical state: the last one wins, the other is not told, and both believe they are in control. This needs ownership.
- A conversion request is short-lived and self-contained. Nothing accumulates between requests and nothing is exclusive. This needs availability, not ownership.

## 17.2 Declaring an access model

```

"rx":    { "channels": [ { "id":"a", "access":"exclusive" } ] } // multi-channel
"rx":    { "access":"exclusive" }                               // single-channel

```

```
"codec": { "access": "shared" }
```

Access control is declared at the level that owns the resource: per channel for a multi-channel rx or tx block, at block level for a single-channel one, and at block level for codec.

For exclusive resources, one client at a time holds control. `open_stream` either succeeds or fails with `in_use`. A device MAY offer an observer role: receive the stream and read status, but not reconfigure. For shared resources no ownership is held; requests are served or refused with `busy`.

Control SHALL be released explicitly on `close_stream`, and implicitly on loss of the control connection.

### 17.3 Reporting the current state

A device implementing `get_status` SHALL report `control_state` for each exclusive resource, with the value `"owned_by_me"`, `"owned_by_other"` or `"free"`. It is reported over the control plane rather than advertised in discovery because it changes on a timescale the mDNS cache cannot follow.

### 17.4 Listening in is a feature

Two operators sharing one receiver while one of them tunes is an ordinary thing to want. That is why the observer role exists rather than a flat refusal. It is MAY rather than SHALL because a microcontroller with a single UDP socket cannot send a second stream.

### 17.5 Capacity

There is deliberately no field in capabilities stating how many concurrent streams a device supports. It is not one number — a codec service with a vocoder chip and a software codec has two independent resources — and it would not be usable, because a client wants to know how much is free right now, which a static field cannot supply.

A device SHALL return `busy` when it cannot serve a request because of resource limits. Clients SHALL treat `busy` as retryable and SHALL NOT infer capacity from capabilities. `stream_already_open` remains for the case where the resource genuinely is one thing.

## 18. Status Reporting [RX-T2 • CODEC-T2]

Purpose. Query the current operational state: stream activity, current settings, access state, and health. Clients use it for status displays, level meters and control indicators. At Tier 1, return `unknown_command`.

### 18.1 Response structure

The result contains one object per implemented function, keyed by function identifier — mirroring the structure of `get_capabilities`.

```
{ "type": "response", "id": "st-1", "status": "ok", "result": {
  "rx": {
    "enabled"           : true,
    "streaming"         : true,
    "control_state"     : "owned_by_me",
    "frequency_hz"      : 14100000,
    "sample_rate_sps"   : 2048000,
    "actual_sample_rate_sps" : 2048000,
    "reference_level_dbm" : -30.0,
    "adc_overload"      : false,
    "adc_level_dbfs"    : -18.4,
    "lo_locked"         : true,
    "controls": { "rf_gain": 20.0, "agc": false, "antenna": "rx1",
                  "ppm_correction": 0, "direct_sampling": "off" },
    "connectors": { "rx1": { "bias_tee": 5.0 },
                   "rx2": { "bias_tee": 0.0 } },
    "hardware_monitors": {
      "pa_temperature_celsius" : null,
      "pa_supply_voltage_v"    : null,
      "pa_drain_current_ma"    : null,
      "board_temperature_celsius": 41.5,
      "supply_voltage_v"       : 5.02,
      "total_current_ma"       : 480
    }
  }
}
```

```

    }
  },
  "tx": {
    "enabled"           : true,
    "streaming"         : false,
    "control_state"     : "owned_by_me",
    "frequency_hz"      : 438500000,
    "sample_rate_sps"   : 1024000,
    "actual_sample_rate_sps" : 1023999,
    "power_dbm"         : null,
    "controls": { "rf_gain": 0, "bias_tee": 0.0 }
  }
}

```

On a multi-channel device the per-channel fields appear inside a `channels` object keyed by channel identifier, following the same rule as capabilities. On a path that declares `connectors[]`, the `connectors` object is keyed by connector id and holds the current values of the controls declared with that connector.

## 18.2 Behaviour

Fields listed as nullable SHALL be present with a JSON null value when unavailable. They SHALL NOT be absent. This lets clients parse unconditionally.

The `controls` object reflects the currently applied state, not the last requested state. If AGC is active and the hardware has changed the gain autonomously, `controls.rf_gain` reflects the hardware.

Controls declared with a connector (§12.5) are reported under `connectors`, never in the flat `controls` object: two `bias_tee` values cannot share one key, and a client that does not know about connectors would otherwise read the wrong one as "the" bias tee.

The nullable fields on the `tx` block are `actual_sample_rate_sps`, `power_dbm` and `control_state`. They are present with null when unavailable — `power_dbm` on a transmitter without `set_tx_power`, `control_state` on a shared transmitter — exactly as on rx.

`actual_sample_rate_sps` matters more than it looks. Some hardware cannot achieve every requested rate exactly. Clients SHALL use `actual_sample_rate_sps`, when it is non-null, for all frequency scale calculations. Clients SHALL use the nominal `sample_rate_sps` for anything that must divide exactly — a resampler ratio, a packet cadence — because 1023999 has no useful factors and 1024000 does.

`enabled` on the `tx` block means the RF path can radiate. A transmitter whose RF output is administratively disabled reports `enabled: false` and `start_tx` succeeds without radiating.

## 18.3 hardware\_monitors [RX-T2]

A named-channel framework for hardware sensors. Six channels are standard, each a number or null:

| Channel                                | Meaning                          |
|----------------------------------------|----------------------------------|
| <code>pa_temperature_celsius</code>    | Power amplifier temperature.     |
| <code>pa_supply_voltage_v</code>       | Power amplifier supply voltage.  |
| <code>pa_drain_current_ma</code>       | Power amplifier drain current.   |
| <code>board_temperature_celsius</code> | Main PCB temperature.            |
| <code>supply_voltage_v</code>          | Main board supply voltage.       |
| <code>total_current_ma</code>          | Total board current consumption. |

The object SHALL always be present at Tier 2 and above with all six channels, null where the sensor is not fitted; the `pa_*` channels are null on hardware with no PA sensor, which is every receive-only device. Clients SHALL ignore channel names they do not recognise. A device with sensors that do not match these names (a tuner die temperature, an LNA thermistor) adds them under prefixed names alongside, and does not report them under a standard name they do not fit. The three board channels were reserved in Release 1.2 and are promoted here because both reference daemons had a board power monitor and nowhere to put it.

### TIP

All six channels as null is a valid response. You do not need any sensors to implement `get_status` at Tier 2.

## 19. Reference Level and Power Display [RX-T2]

The Reference Level converts I/Q sample amplitude to absolute received power at the antenna port, in dBm. Without it a client can display relative levels; with it, the S-meter becomes a calibrated instrument.

It answers one question: if a full-scale sample were produced, what power at the antenna would have caused it? A value of -30.0 dBm means a -30 dBm signal produces samples at maximum amplitude.

```
P_dBm = ReferenceLevel_dBm + 20 * log10( RMS_amplitude / full_scale )

full_scale: 32767 (int16)  127 (u8)  8388607 (int24)  1.0 (float32)
RMS for complex I/Q : sqrt( mean(I^2 + Q^2) )
RMS for real          : sqrt( mean(I^2) )

Example:
ReferenceLevel = -30.0 dBm
Measured RMS   = 3277 counts (int16) = 0.1 of full scale
Level in dBFS  = 20 * log10(0.1) = -20.0 dBFS
P_dBm          = -30.0 + (-20.0) = -50.0 dBm
```

For an unsigned encoding the client subtracts the stream's zero\_point (SDR-SLC-VITA-1.0 §6.5) from every sample before computing the RMS, so that the converter's real midpoint, not the nominal 128, is treated as zero signal.

The Reference Level is reported through three mechanisms. A client should use the most authoritative one available.

| Mechanism                                             | Authority            | When to use                                                                                                                                  |
|-------------------------------------------------------|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| capabilities.rx.reference_level_dBm_at_preferred_gain | Low — static         | Read once at startup, before streaming.                                                                                                      |
| get_status → reference_level_dBm                      | Medium — current     | Reflects current gain. Poll after gain changes if not parsing Context packets.                                                               |
| VITA-49 Context packet Reference Level field          | High — authoritative | In-band, real time. Updates on every gain change while streaming. Encoded per SDR-SLC-VITA-1.0 Release 1.2 §7.4 (VITA-49.0 Q9.7, 1/128 dBm). |

The Reference Level tracks gain automatically. When the operator raises rf\_gain by 10 dB, the device lowers the Reference Level by 10 dB.

|                |                                                                                                                                                                                                                                                                                                                                                                                                    |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>WARNING</b> | Reference Level is a nominal calibration. An uncharacterised RTL-SDR dongle: plus or minus 5 to 8 dB absolute accuracy. A device with per-frequency, per-gain lookup tables: plus or minus 1 to 2 dB. A device <b>SHOULD</b> publish reference_level_accuracy_db in capabilities.rx; clients <b>SHOULD</b> display it and <b>SHOULD NOT</b> present uncalibrated readings as precise measurements. |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

### 19.1 Tier 2 capability hints

| Field                                 | Description                                                                                                                                                                                                                           |
|---------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| preferred_sample_rate_sps             | The rate at which the hardware performs best. The client uses it as its default. If absent, the client uses the first entry in sample_rates_sps. Defined for the tx block with the same meaning.                                      |
| preferred_gain_db                     | Suggested starting gain for typical reception. The client pre-populates the gain control with it.                                                                                                                                     |
| reference_level_dBm_at_preferred_gain | Power at the antenna corresponding to a full-scale sample at preferred_gain_db.                                                                                                                                                       |
| reference_level_accuracy_db           | Nominal absolute accuracy of the Reference Level, in dB.                                                                                                                                                                              |
| usable_bandwidth_hz                   | Optional. Flat passband width in Hz, narrower than the sample rate. Clients use it to place slices and to grey out the roll-off at the edges of the waterfall; a client that does not see it assumes the whole sample rate is usable. |

Where two function blocks share a sample clock (§12.4), a device **SHOULD** advertise the same preferred\_sample\_rate\_sps for both, so that a client following the hints does not require a rate change on one while the other is streaming. A device **MAY** still support different rates on request.

## 20. Clock and Location [RX-T2 query • RX-T3 set]

Accurate VITA-49 timestamps depend on a synchronised device clock. The source is reported by `get_clock_status`.

| Source                                   | Accuracy                             | Notes                                                                                                           |
|------------------------------------------|--------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| PTPv2 grandmaster (GPS-locked)           | < 1 $\mu$ s                          | Best. UTC-traceable.                                                                                            |
| PTPv2 grandmaster (non-GPS)              | < 100 $\mu$ s                        | Good. Depends on grandmaster quality.                                                                           |
| GPS receiver on device                   | < 1 $\mu$ s                          | Good. Requires an on-device GPS module.                                                                         |
| Manual ( <code>set_clock</code> )        | Client accuracy                      | Acceptable for logging. Not UTC-traceable.                                                                      |
| Host clock ( <code>free_running</code> ) | Host NTP accuracy, typically 1–50 ms | The device stamps its host's best estimate of UTC, <code>synced: false</code> . What both reference daemons do. |
| None (no time source)                    | Relative only                        | Integer timestamps count from stream start with TSI=11.                                                         |

A device SHALL stamp the VITA-49 Timestamp Integer with its best estimate of UTC under TSI=01 whenever it has any time source at all, including an undisciplined host clock, and SHALL report the source and `synced` honestly in `get_clock_status`. Only a device with no time source whatsoever counts seconds from `start_rx`, and it says so on the wire with TSI=11 (SDR-SLC-VITA-1.0 §6.4.1). Clients SHOULD warn the operator when `synced: false`.

This replaces the Release 1.2 rule, under which a free-running device counted from stream start. Both shipped devices stamped UTC anyway, the client relies on it to align two receivers, and a relative counter throws that away for no gain.

```

C->S { "type":"request","id":"ck-1","command":"get_clock_status","params":{} }
S->C { "type":"response","id":"ck-1","status":"ok","result":{
  "source"      : "free_running",
  "synced"      : false,
  "accuracy_us" : null,
  "utc_time"    : "2026-09-14T09:12:01.000Z",
  "epoch_s"     : 1789377121,
  "ptp"         : { "available":false },
  "gps"         : { "available":false }
} }

```

### 20.1 Location [RX-T3]

`get_location` and `set_location` manage the operator's Maidenhead locator. Maidenhead SHALL use the 6-character sub-square format with lowercase sub-square characters, for example J022ki and not J022KI. This is embedded in VITA-49 GPS Context packets when available.

## 21. Asynchronous Events

Events are device-generated messages a client receives without issuing a request. They report state changes: stream status, frequency changes, gain adjustments, ADC overload, hardware alerts.

### 21.1 Naming and scope

Event names in the reserved registry below are unprefixes. Events belonging to a function block SHALL be prefixed with the block name, for example `codec.state`. Vendor-defined events SHALL be prefixed with a vendor identifier (§3.5).

An event relating to a specific function SHALL carry a `function` parameter identifying it, and a `channel` parameter where the device has more than one channel. Clients SHALL silently ignore event names they do not recognise.

### 21.2 Event registry

| Event                            | Tier          | Trigger                                 | Key parameters                                                                                |
|----------------------------------|---------------|-----------------------------------------|-----------------------------------------------------------------------------------------------|
| <code>stream_state</code>        | RX-T1 • TX-T1 | Stream started, stopped or failed.      | <code>stream_id</code> , <code>streaming</code> , <code>function</code> , <code>reason</code> |
| <code>frequency_changed</code>   | RX-T2         | Synthesiser settled on a new frequency. | <code>function</code> , <code>channel</code> , <code>frequency_hz</code>                      |
| <code>sample_rate_changed</code> | RX-T2         | Sample rate changed.                    | <code>function</code> , <code>sample_rate_sps</code> , <code>actual_sample_rate_sps</code>    |

| Event              | Tier     | Trigger                                                           | Key parameters                                                     |
|--------------------|----------|-------------------------------------------------------------------|--------------------------------------------------------------------|
| control_changed    | RX-T2    | A control value changed, including AGC-driven changes.            | function, name, value, connector (§12.5, when declared)            |
| adc_overload       | RX-T2    | ADC clipping detected. Emitted at onset, not continuously.        | stream_id, adc_level_dbfs, suggested_action                        |
| clock_sync_changed | RX-T2    | PTP or GPS sync state changed.                                    | source, synced, accuracy_us                                        |
| warning            | RX-T2    | Non-fatal device condition.                                       | code, message                                                      |
| control_revoked    | ANY      | An exclusive resource was taken or released.                      | function, channel, connector (§12.5, when declared), control_state |
| tx_burst_late      | TX-T2    | A scheduled burst arrived after its start sample and was dropped. | stream_id, samples_late                                            |
| rx_epoch_changed   | TX-T2    | The device reset its rx sample counter under an open stream.      | rx_stream_epoch_sample                                             |
| vswr_alert         | TX-T3    | VSWR exceeded threshold during transmit.                          | vswr, threshold, tx_reduced                                        |
| monitor_alert      | TX-T3    | A hardware monitor channel crossed a threshold.                   | channel, value, threshold, tx_inhibited                            |
| gps_fix_changed    | RX-T3    | GPS fix acquired or lost.                                         | fix, satellites, maidenhead                                        |
| tuner_state        | TX-T3    | Antenna tuner cycle completed.                                    | tuned, vswr_after                                                  |
| codec.state        | CODEC-T1 | Codec instance started, stopped or failed.                        | stream_id, state                                                   |

stream\_state.reason is optional and takes one of: client\_request (a start or stop command), turnaround (auto turnaround muted or resumed the other direction), tx\_data\_timeout and control\_lost (§15.7), monitor\_inhibit (§21.4), device\_error. A client SHALL ignore a value it does not know. It exists so that a client whose PA just dropped can tell the operator why.

### 21.3 adc\_overload

Emitted when the ADC starts clipping. Clipping produces spurious signals on the waterfall that look like real ones, so the client should show a prominent warning and recommend reducing gain.

```
{ "type":"event", "event":"adc_overload", "params":{
  "stream_id"      : "rx-1",
  "adc_level_dbfs" : -0.3,
  "suggested_action" : "reduce_rf_gain"
} }
```

### 21.4 monitor\_alert

Emitted when a hardware\_monitors channel crosses a warning or alarm threshold. tx\_inhibited is true when the device has autonomously stopped transmitting to protect itself.

```
{ "type":"event", "event":"monitor_alert", "params":{
  "channel"      : "pa_temperature_celsius",
  "value"        : 81.2,
  "threshold"    : "alarm",
  "threshold_value" : 80.0,
  "tx_inhibited" : true
} }
```

#### WARNING

When tx\_inhibited is true, the device has stopped transmit output on its own. The client SHALL reflect this and SHALL NOT attempt start\_tx until a subsequent monitor\_alert reports tx\_inhibited:false.

## 21.5 Subscriptions

There is deliberately no subscribe or unsubscribe command in this release. A device that produces events continuously would need subscriptions and flow control, but no function block defined here does, and §23 guarantees that both can be added later without breaking anything.

## 22. Error Code Registry [ENVELOPE]

Every error response SHALL use one of these codes in `error.code`. `error.message` is a human-readable string for display and logging.

| Error code                      | When to use                                                                                                                           |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| unknown_command                 | Command name not recognised. Return this for anything not in your implementation.                                                     |
| invalid_request                 | Malformed JSON, or a non-request message where a request was expected, or any command before hello.                                   |
| missing_parameter               | A required field is absent from params.                                                                                               |
| invalid_parameter               | A field is present but has an invalid value or the wrong type. Name the field in <code>error.message</code> .                         |
| unsupported_protocol_version    | No version in the client's hello list is supported.                                                                                   |
| unsupported_frequency           | Frequency outside all ranges published for the addressed function.                                                                    |
| unsupported_sample_rate         | Rate not in the published <code>sample_rates_sps</code> list.                                                                         |
| unsupported_format              | I/Q format not in <code>iq_formats</code> , or its supported flag is false.                                                           |
| unsupported_stream_protocol     | Requested transport protocol not present in the function's <code>transports[]</code> .                                                |
| unsupported_conversion          | Requested from/to pair not present in the <code>conversions[]</code> list.                                                            |
| unsupported_power               | Transmit power outside the range from <code>get_tx_capabilities</code> .                                                              |
| stream_not_found                | <code>stream_id</code> does not match any open session.                                                                               |
| stream_already_open             | A stream is already open on a resource that supports only one.                                                                        |
| stream_not_open                 | A start command arrived with no stream open for that function.                                                                        |
| stream_reconfiguration_required | The requested change requires closing and reopening the active stream, or another block in the same clock group is streaming (§12.4). |
| in_use                          | An exclusive resource is held by another client.                                                                                      |
| unauthorized                    | The addressed function requires authentication and the connection has not authenticated.                                              |
| auth_failed                     | An authenticate attempt was rejected: unknown method, unknown key identifier, or incorrect response.                                  |
| busy                            | The device cannot serve the request because of resource limits. The client should retry after a short delay.                          |
| tune_failed                     | <code>auto_tune</code> could not reach the target VSWR. <code>details.best_vswr</code> carries the achieved value.                    |
| clock_sync_active               | <code>set_clock</code> rejected because an active sync source is present.                                                             |
| device_error                    | Hardware or DSP error. Include diagnostic codes in <code>error.details</code> if available.                                           |
| not_supported                   | The feature is not implemented on this hardware. Use for functions the device does not have and controls it does not expose.          |
| timeout                         | A hardware operation timed out.                                                                                                       |
| internal_error                  | Unexpected firmware error. Reconnect recommended; the device may be in an undefined state.                                            |

|             |                                                                                                                                              |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| <b>NOTE</b> | No codes are added in Release 1.3. A device SHALL NOT invent codes; <code>error.details</code> is the place for device-specific information. |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------|

## 23. Extensibility and Forward Compatibility [ENVELOPE]

SLC-CP is designed to grow without breaking existing implementations. These rules govern how.

| Scenario                                                  | Correct behaviour                                                                                          |
|-----------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| Client sends a field the device does not know             | Device silently ignores the unknown field.                                                                 |
| Device sends a field the client does not know             | Client silently ignores the unknown field.                                                                 |
| Client sends an unknown command                           | Device returns unknown_command. The connection stays open.                                                 |
| Client receives an unknown event                          | Client silently ignores it.                                                                                |
| Client sees an unknown function block in capabilities     | Client ignores that block and uses the ones it knows.                                                      |
| Client sees an unknown function in a discovery TXT record | Client still shows the instance. It does not reject it.                                                    |
| Client sees an unknown control type                       | Client ignores that descriptor. It does not abort the session.                                             |
| Client sees a connector it does not know                  | Client ignores that descriptor. Controls without a connector are unaffected.                               |
| Client sees an unknown format in formats[]                | Client ignores that format, and any conversion referring to it.                                            |
| Client sees an unknown authentication method              | Client uses a method it knows, or reports that it cannot authenticate.                                     |
| Client sees an unknown stream_state.reason                | Client ignores the reason and acts on streaming.                                                           |
| New controls added to controls[]                          | Clients that do not know a control simply do not set it.                                                   |
| New iq_formats entries added                              | Added with supported:false until the firmware implements them. Clients check the flag before requesting.   |
| New error codes added                                     | Clients treat unrecognised codes as internal_error.                                                        |
| schema_version higher than the client knows               | Client processes what it recognises and ignores the rest.                                                  |
| Protocol minor version increment                          | Backward compatible. Adds optional fields, commands and events. Existing implementations continue to work. |
| Protocol major version increment                          | Breaking change. Requires new hello version negotiation.                                                   |

### WARNING

Release 1.3 breaks Release 1.2 in the four places listed in §1.2, because Protocol 1.0 has not been declared final and a release candidate is not a stable interface. Once Protocol 1.0 FINAL is published, the table above applies without exception, and a change of that kind would require Protocol 2.0.

### 23.1 Reserved field names

These names are reserved for future revisions. Do not use them for other purposes in vendor extensions.

| Reserved name                  | Location                 | Intended future use                                           |
|--------------------------------|--------------------------|---------------------------------------------------------------|
| noise_figure_db                | capabilities.rx          | Receiver noise figure in dB at the antenna port.              |
| ip3_dbm                        | capabilities.rx          | Input third-order intercept point in dBm.                     |
| sfdr_db                        | capabilities.rx          | Spurious-free dynamic range in dB.                            |
| gain_compression_1db_dbm       | capabilities.rx          | 1 dB gain compression input power.                            |
| phase_noise_dbc_hz             | capabilities.rx          | LO phase noise at 1 kHz offset.                               |
| filter_bandwidth_3db_hz        | capabilities.rx          | IF filter 3 dB bandwidth.                                     |
| clock_discipline               | capabilities             | Declared clock quality level.                                 |
| oscillator_error_ppm_estimated | get_status rx            | Device-estimated oscillator error in PPM.                     |
| frequency_correction_source    | get_status rx            | Source of PPM correction: manual, gps, factory or auto.       |
| tuner_temperature_celsius      | get_status rx            | Tuner IC die temperature.                                     |
| lna_temperature_celsius        | hardware_monitors        | External LNA temperature.                                     |
| slices                         | capabilities.rx.channels | Device-side extraction of narrow streams from a wide channel. |

## 24. Conformance Requirements

An implementation claiming a function block at a tier SHALL satisfy the envelope requirements plus all requirements for that block up to and including that tier.

### 24.1 Envelope — every device

- Implement TCP transport with 4-byte big-endian length-prefixed JSON framing.
- Implement hello: validate `supported_protocol_versions` and return the selected version.
- Reject any command received before a successful hello with `invalid_request`.
- Implement `get_capabilities` returning `schema`, `schema_version` as defined by this specification, and a key for each implemented function block.
- Omit the capability key of any function block not implemented.
- Return `unknown_command` for every unimplemented command.
- Silently ignore unknown JSON fields in every received message.
- Treat all `_hz` fields as 64-bit unsigned integers.
- Stop all streams and release all sessions on TCP connection loss.
- Not reuse stream IDs within a TCP session.
- Use only error codes from §22.
- Prefix every vendor-defined name (§3.5).
- Advertise over mDNS with a functions TXT field listing implemented blocks and an `fw` field of the form `<implementation>/<version>`.

### 24.2 Block rx, Tier 1

- Publish `capabilities.rx` with `tier`, `access`, `frequency_ranges_hz`, `sample_rates_sps`, `transports`, `iq_formats`, and `controls` containing at least `rf_gain` with a `type` field.
- Publish sample rates as a discrete list only. Do not additionally publish a minimum/maximum range.
- Implement `set_frequency`, `set_sample_rate`, and `set_control` for `rf_gain`.
- Implement `open_stream` with `protocol: "vita49"` and transport parameters inside a `vita` object; return `signal_type`.
- Implement `start_rx`, `stop_rx`, `close_stream`.
- Send VITA-49 packets to the destination given in `open_stream`. Never assume a client port.
- Emit `stream_state` with a `function` field on start and stop.
- Return `not_supported` for `set_control` names other than those published.
- If the device has more than one signal path: publish `channels`, and declare each control at the level where the hardware shares it. If it has one: omit `channels` entirely.
- If a path has more than one physical port with hardware of its own: publish `connectors[]` on that block or channel, declare the antenna control over the connector ids, and put `connector` on every control that belongs to one port. If one: omit `connectors[]` and never send `connector`.
- If the sample clock is shared with another block: declare `shares_sample_clock_with`.

### 24.3 Block rx, Tier 2

- All Tier 1 requirements.
- Implement `ping`, `get_status`, `get_control`, `get_clock_status`.
- Report `actual_sample_rate_sps`, `reference_level_dbm`, `adc_overload`, `adc_level_dbfs`, `lo_locked`, using JSON null where the hardware cannot supply a value.
- Report `control_state` for every exclusive resource.
- Include the `hardware_monitors` object with all six standard channels, null where not fitted.
- Publish `preferred_sample_rate_sps`, `preferred_gain_db`, `reference_level_dbm_at_preferred_gain`.

- Implement the standard controls the hardware has: `agc`, `bias_tee`, `ppm_correction`, `direct_sampling`; return `not_supported` for the others.
- Emit a VITA-A/1.0-CTX Context packet, encoded per SDR-SLC-VITA-1.0 Release 1.2, before the first IF Data packet after `start_rx`, after any gain change, and after a signal mode change.
- Emit `frequency_changed`, `sample_rate_changed`, `control_changed`, `adc_overload`, `clock_sync_changed`, `warning`.
- Stamp UTC under `TSI=01` when any time source exists (§20).
- Not use reserved field names for other purposes.

## 24.4 Block tx, Tier 1

- Publish `capabilities.tx` with `tier`, `access`, `full_duplex`, `frequency_ranges_hz`, `sample_rates_sps`, `transports` including `listen_port`, `iq_formats`, `tx_data_timeout_ms`, and `controls` for every control the transmitter exposes. Publish `turnaround_mode` unless `full_duplex` is true, in which case it MAY be omitted.
- If the tx path has more than one physical port with hardware of its own: publish `connectors[]` and per-connector controls as in §24.2. If one: omit `connectors[]`.
- Implement `get_tx_capabilities`.
- Support function: "tx" on `set_frequency`, `set_sample_rate`, `open_stream`, and on `set_control` for every control published in `capabilities.tx.controls`.
- Return `listen_port`, `stream_id`, `samples_per_packet`, `format` and `signal_type` from `open_stream`.
- Implement `start_tx`, `stop_tx`.
- Emit `stream_state` with function: "tx" and a reason on start and stop.
- For auto turnaround: emit `stream_state` for both functions on start and stop.
- Return `not_supported` for `get_vswr`, `auto_tune` and `set_tx_power` unless implemented; `unknown_command` for `get_timestamps` unless implemented.
- Stop transmitting on loss of the control connection, and when no transmit data has arrived within the device timeout; emit `stream_state` with the reason.
- Drop transmit output before dropping bias tee power; restore bias tee power before resuming output.
- In `get_status`, report the nullable tx fields `actual_sample_rate_sps`, `power_dbm` and `control_state` present with null when unavailable.

## 24.5 Block tx, Tier 2

- All Tier 1 requirements.
- Implement `set_tx_power` addressed by function and publish the power range, or publish `rf_gain` in `capabilities.tx.controls`; at least one of the two.
- Implement `get_timestamps` and advertise `protocol_version: "VITA-A/1.0-TX"`.
- Accept VITA-A/1.0-TX packets with scheduled burst start and burst end; also accept plain VITA-A/1.0 packets as continuous transmission.
- Drop a late burst and emit `tx_burst_late`.

## 24.6 Block codec, Tier 1

- Publish `capabilities.codec` with `tier`, `access`, `transports` including `rtp`, `formats` and `conversions`.
- Give every format a unique `id` and a `kind`, and describe it fully; give every frame format `frame_bits` consistent with `rate_bps` and `frame_ms`.
- List every performable conversion as a from/to pair referring to those identifiers.
- Implement `open_stream` with `protocol: "rtp"`, `from` and `to`.
- Bind an RTP receive port for the input side and return it in `rtp.input.receive` with a reachable unicast address.
- Send output to `rtp.output.destination`. Never assume it belongs to the controlling client.
- Implement `start_codec`, `stop_codec`, `close_stream`.

- Return `unsupported_conversion` for a pair not present in `conversions[]`.
- Return `busy` when resource limits prevent serving a request.
- Advertise over mDNS on the `_slc-codec._tcp` service type.

## 24.7 Clients, any tier

- Send hello first, offering only "1.0", and verify `selected_protocol_version` before proceeding.
- Call `get_capabilities` before any configuration command.
- Correlate responses to requests by `id`. Do not assume response ordering. Track every outstanding request.
- Treat all `_hz` fields as 64-bit unsigned integers. Use `BigInt` in JavaScript.
- Silently ignore unknown JSON fields, unknown events, unknown function blocks, unknown codec entries and unknown `stream_state.reason` values.
- Treat the absence of a capability key as the absence of that function, not as an error.
- Handle `unknown_command` and `not_supported` by disabling the corresponding control, not by showing the operator an error; stop sending a command that returned `unknown_command`.
- Not make behavioural decisions based on a declared tier.
- Render `controls[]` of every block the device declares, addressing `set_control` to that block.
- Render a control that carries `connector` once per declared connector, labelled with that connector's `label`; send `connector` on `set_control` and `get_control` for those controls and never for the others.
- Send `bias_tee` values from `allowed_values` only.
- Use `actual_sample_rate_sps`, when non-null, for all frequency scale calculations, and the nominal rate for exact ratios.
- Not issue `set_sample_rate` to a block whose clock group has an active stream (§12.4).
- Treat the VITA-49 Context packet Reference Level as authoritative over `reference_level_dbm`.
- Silently discard VITA-49 packets with Packet Type 0x4 if Context parsing is not implemented.
- Not infer IF Data packet loss from Context packet counts.
- Re-run the full startup sequence after any TCP disconnection.
- Treat `busy` as retryable and surface `in_use` to the operator. Do not infer capacity from capabilities.
- Authenticate before addressing a function whose `auth` is not `none`, and handle `unauthorized` by prompting rather than failing.
- Inhibit `start_tx` while a `monitor_alert` reports `tx_inhibited`.
- Re-read `get_timestamps` after `rx_epoch_changed`.

## 25. Security Considerations

SLC-CP provides no encryption and, by default, no authentication. It is designed for operation on a trusted local network. On a device that has not enabled the optional authentication of §26, any host that can reach TCP port 4620 can issue every control command, including tuning, changing gain, and starting a transmission.

### WARNING

Do not expose an SDR-SLC device to the Internet without a firewall. Any host that reaches port 4620 can start a transmit stream on your amateur radio station. Network-level access controls — firewall rules, VLAN isolation — are the correct mitigation.

### 25.1 Ownership is not a security mechanism

The ownership model in §17 exists to stop two cooperating clients from confusing each other. It stops nothing else. Any host on the network can claim an exclusive resource and hold it. Do not read `in_use` as an access control decision; access control is §26, and it is a separate mechanism with separate errors.

### 25.2 A network of services increases exposure

A station now normally runs several SDR-SLC devices, each listening on a control port and each accepting media on a UDP port negotiated at runtime. Two practical consequences. First, a device **SHOULD** accept media only from the peer

address negotiated in `open_stream` (`vita.source` for transmit, `rtp.input.expect_source` for codec), which costs one comparison and removes the easiest way to inject data into a running stream; a client **SHOULD** do the same for the packets it receives (SDR-SLC-VITA-1.0 §13). Second, a device **SHOULD** bind its listening sockets to a specific interface rather than to all of them.

## 25.3 Transmitters

A networked transmitter is not merely another device on the network. In most jurisdictions the licence holder is answerable for everything radiated from the station, whoever caused it, so an unprotected transmit function is a liability rather than an inconvenience. §26 defines an authentication method for exactly this case, and §15.7 requires a transmitter to stop of its own accord when its client goes away.

## 25.4 Future work

A future version **MAY** introduce TLS 1.3 wrapping for the control connection, providing encryption and optional mutual certificate authentication. The length-prefixed framing is fully TLS-compatible without any change to protocol structure.

# 26. Authentication [ANY]

Authentication is optional to implement. A device may implement it, and an operator may switch it on for the functions that need it. A receiver has no particular reason to use it. A transmitter does: in most jurisdictions the licence holder is personally answerable for everything radiated from their station, whoever pressed the button.

## 26.1 Authentication is declared per function

Each function block declares its own requirement, alongside the access model of §17.

```
"rx": { "access": "exclusive", "auth": "none" },
"tx": { "access": "exclusive", "auth": "psk-hmac-sha256" },
"codec": { "access": "shared", "auth": "none" }
```

A function block **MAY** declare an `auth` field naming the authentication method required to address it. A device that omits the field for a block **SHALL** treat that block as requiring no authentication. A device **SHALL** return unauthorized for any command addressing a function whose `auth` is not `none` on a connection that has not successfully authenticated for it.

Authentication **SHALL NOT** be required for `hello` or `get_capabilities`. A client must be able to discover what a device is, and what it would need in order to use it, before authenticating.

A device that implements the `tx` block **SHALL** implement authentication and **SHALL** require it for the `tx` block by default. The operator may switch it off. Release 1.2 said **SHOULD**; both reference daemons then shipped with it off, which is the outcome that sentence was meant to prevent. Optional to enable is not the same as optional to implement.

## 26.2 Method: psk-hmac-sha256

One method is defined. It is a challenge-response over a pre-shared key, chosen because it fits on a microcontroller: HMAC-SHA256 is a few hundred lines, or a hardware block that an STM32H7 already has.

Challenge. When any function of the device requires authentication, the `hello` response carries a challenge:

```
S->C { "type": "response", "id": "h-1", "status": "ok",
      "result": { "protocol_name": "SDR-SLC Control Plane",
                  "selected_protocol_version": "1.0",
                  "device_name": "SLC-BLADERF-01",
                  "manufacturer": "Nuand",
                  "auth": {
                      "methods": [ "psk-hmac-sha256" ],
                      "nonce": "9f2c41ab7de05318c4a6b90f2e71dd83"
                  }
      } }
```

A device SHALL include the auth object in the hello response if and only if at least one of its function blocks requires authentication. The nonce SHALL be at least 16 octets, hex-encoded, freshly generated for each connection, and not predictable from previous nonces.

Response.

```
C->S { "type":"request", "id":"a-1", "command":"authenticate",
      "params":{"method":"psk-hmac-sha256",
                "key_id":"shack-laptop",
                "response":"1b8e...c07f" } }
S->C { "type":"response", "id":"a-1", "status":"ok",
      "result":{"authenticated":true, "functions":["tx"]} }
```

| Field    | Type   | Required | Description                                                                                                                                                       |
|----------|--------|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| method   | string | YES      | Method name from the methods array in the hello challenge.                                                                                                        |
| key_id   | string | NO       | Identifies which pre-shared key was used. Defaults to "default". Lets a device hold several keys so that one client can be revoked without disturbing the others. |
| response | string | YES      | Hex-encoded MAC, computed as described below.                                                                                                                     |

The result reports which functions the connection has unlocked.

Computing the response.

```
nonce_bytes = hex_decode( auth.nonce )
mac          = HMAC-SHA256( psk, nonce_bytes )
response     = hex_encode_lowercase( mac )
```

The pre-shared key SHALL be used as the HMAC key exactly as configured, as UTF-8 octets, without transformation. The message SHALL be the decoded nonce octets and nothing else. Comparison of the received MAC SHALL be performed in constant time.

|            |                                                                                                                                                                                                                                                                                                                                                             |
|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>TIP</b> | The whole method is testable by hand: <code>printf '%s' "\$NONCE"   xxd -r -p   openssl dgst -sha256 -hmac "\$PSK"</code> . Take the nonce from a hello response, run that, and send the result. If it does not work you can compare intermediate values with any command-line tool, which is the same reason JSON was chosen for the rest of the protocol. |
|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 26.3 Behaviour

- Authentication is scoped to the TCP session. A new connection starts unauthenticated and receives a new nonce.
- A client MAY call `authenticate` more than once, for example to present a second key. Each successful call adds to the set of unlocked functions; it never removes from it.
- A failed attempt SHALL return `auth_failed` and SHALL NOT reveal whether the method, the key identifier or the MAC was at fault.
- A device SHOULD rate-limit failed attempts, and MAY close the connection after several. Ten attempts per minute is ample for a legitimate client.
- A device SHALL NOT reuse a nonce after a failed attempt on the same connection. Issue a fresh one in the `auth_failed` response's `details.nonce`, or require reconnection.

A device SHALL NOT ship with a pre-configured pre-shared key. A device requiring authentication with no key configured SHALL refuse the protected functions rather than accepting any key. Factory default credentials are the most reliably exploited weakness in consumer network equipment, and a device that transmits is the worst possible place to repeat that mistake.

## 26.4 What this protects, and what it does not

The method authenticates the client to the device at the start of a session. It does not encrypt anything, and it does not authenticate individual messages. An attacker who can observe and modify traffic on the path can wait until a legitimate client has authenticated and then inject commands into the established connection.

On a home network that is the right trade, and it is a large improvement over nothing. Across the public Internet it is not sufficient. Devices implementing the tx block SHALL NOT be exposed directly to the Internet, with or without authentication. Where remote access is required, it SHALL be provided through an encrypted tunnel such as a VPN.

|                |                                                                                                                                                                                                                                                                                             |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>WARNING</b> | Authentication makes a station safe against the household and the guest network. It does not make a transmitter safe to put on a public address, and treating it as though it does is worse than having no authentication at all, because it replaces a visible risk with an invisible one. |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 26.5 Extending the method set

methods is an array so that further methods can be added without disturbing anything. A future TLS-based method would appear alongside this one, and a client would choose the strongest it supports. Clients that do not recognise a method skip it; a client that recognises none reports that it cannot authenticate.

## 27. References

### 27.1 Normative references

| Reference                       | Title                                                | Relevance                                                                                          |
|---------------------------------|------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| SDR-SLC-PS-1.0                  | SDR-SLC Protocol Suite Architecture                  | Overall system architecture, network model, design rationale.                                      |
| SDR-SLC-DISC-1.0                | SLC-DISC Service Discovery                           | mDNS/DNS-SD advertisement. How clients find devices.                                               |
| SDR-SLC-VITA-1.0<br>Release 1.2 | VITA-SLC I/Q Stream Transport                        | VITA-A/1.0, VITA-A/1.0-CTX and VITA-A/1.0-TX packet formats, timestamps, Reference Level encoding. |
| SDR-SLC-RTP-1.0                 | SLC-RTP Framed Media Transport                       | RTP payload formats and packetisation for the codec function block.                                |
| RFC 793                         | Transmission Control Protocol                        | TCP transport.                                                                                     |
| RFC 2119                        | Key Words for Use in RFCs                            | Normative keyword definitions.                                                                     |
| RFC 2104                        | HMAC: Keyed-Hashing for Message Authentication       | Authentication method defined in §26.                                                              |
| RFC 3550                        | RTP: A Transport Protocol for Real-Time Applications | RTP transport used by the codec function block.                                                    |
| RFC 6234                        | US Secure Hash Algorithms                            | SHA-256, used by the authentication method in §26.                                                 |
| RFC 6762                        | Multicast DNS                                        | mDNS protocol used by SLC-DISC.                                                                    |
| RFC 6763                        | DNS-Based Service Discovery                          | DNS-SD used by SLC-DISC.                                                                           |
| ANSI/VITA 49.0                  | VITA Radio Transport Standard                        | Base VRT packet format that VITA-A/1.0 profiles.                                                   |

### 27.2 Informative references

| Reference          | Title                                                         | Notes                                                         |
|--------------------|---------------------------------------------------------------|---------------------------------------------------------------|
| RFC 8259           | The JavaScript Object Notation (JSON) Data Interchange Format | JSON encoding rules.                                          |
| IEEE 1588-2008     | Precision Time Protocol v2                                    | PTPv2 clock synchronisation.                                  |
| IEEE 802 OUI       | OUI Registry                                                  | IXChange BV OUI: 40-28-14.                                    |
| RTL2832U datasheet | Realtek RTL2832U                                              | Clock divider constraints relevant to actual_sample_rate_sps. |
| AD9361 datasheet   | Analog Devices AD9361                                         | Shared RX/TX sample clock; relevant to §12.4.                 |
| sdr-slc-rtld       | Reference implementation, RTL-SDR bridge                      | Block rx, Tier 2.                                             |
| sdr-slc-bladerfd   | Reference implementation, bladeRF 2.0 micro                   | Blocks rx and tx, scheduled bursts.                           |
| cJSON              | Ultralight JSON parser                                        | Recommended JSON library for embedded                         |

| Reference | Title                | Notes                                     |
|-----------|----------------------|-------------------------------------------|
|           |                      | implementations.                          |
| lwIP      | Lightweight IP stack | TCP/IP stack for STM32H7 implementations. |